

Recurrent Neural Networks

Wei Xu

(many slides from Greg Durrett)

This Lecture

- ▶ Recurrent neural networks
- ▶ Vanishing gradient problem
- ▶ LSTMs / GRUs
- ▶ Applications

Readings

- ▶ Reading: RNNs
 - ▶ Eisenstein 7.6
 - ▶ Jurafsky and Martin, Chapter 13
 - ▶ Goldberg 10, 11

A Primer on Neural Network Models for Natural Language Processing

Yoav Goldberg
Draft as of October 5, 2015.

The most up-to-date version of this manuscript is available at <http://www.cs.biu.ac.il/~yogo/nnlp.pdf>. Major updates will be published on arxiv periodically. I welcome any comments you may have regarding the content and presentation. If you spot a missing reference or have relevant work you'd like to see mentioned, do let me know. first.last@gmail

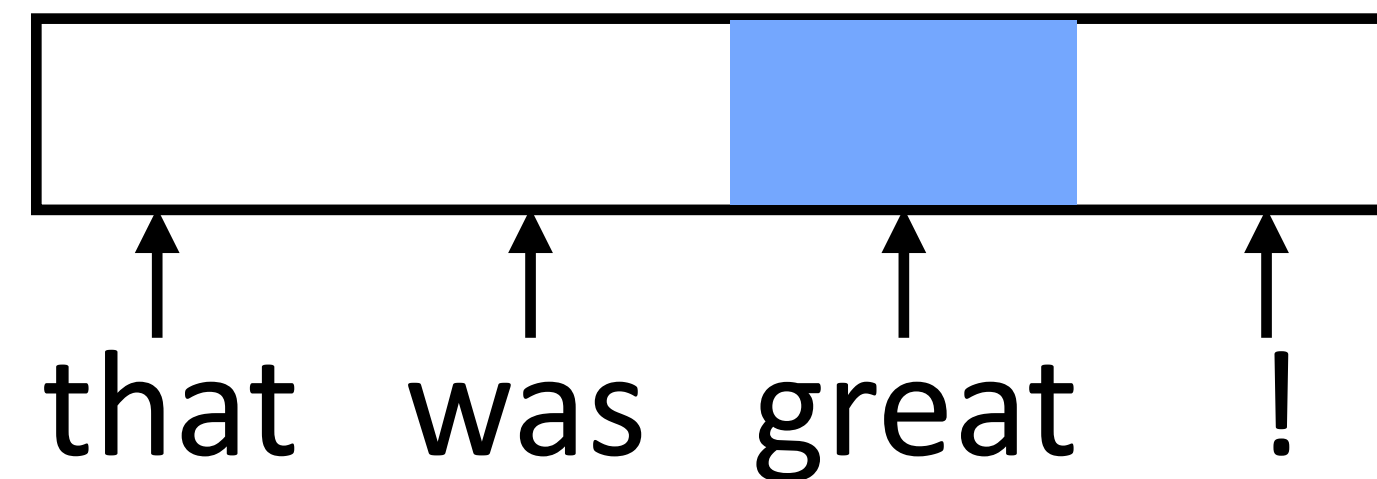
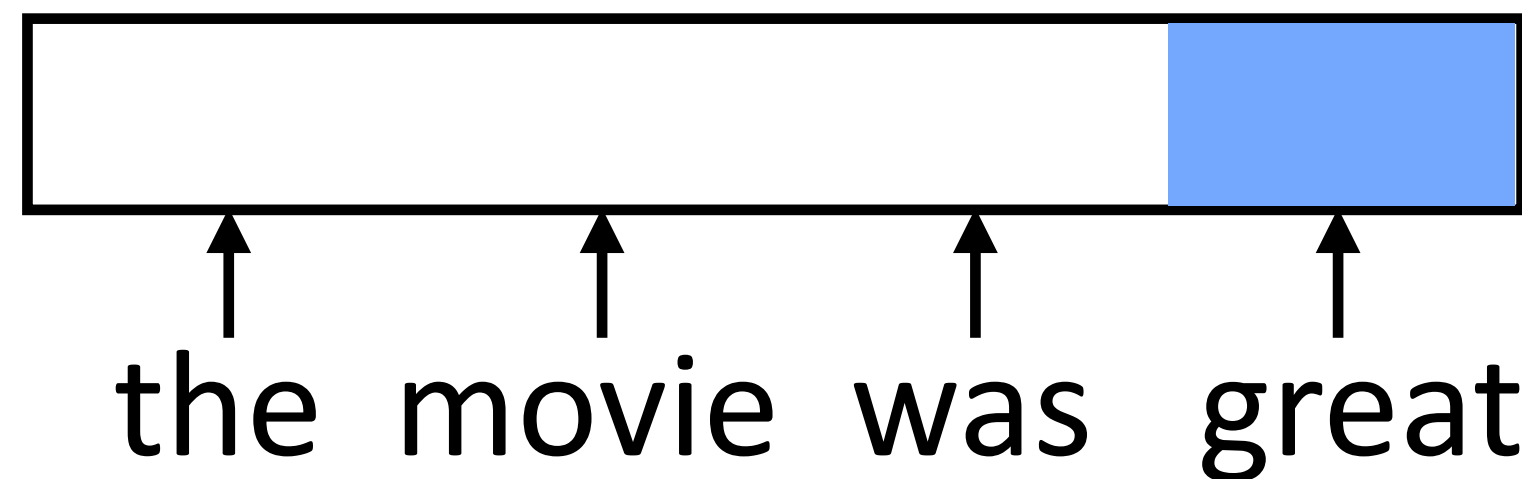
Abstract

Over the past few years, neural networks have re-emerged as powerful machine-learning models, yielding state-of-the-art results in fields such as image recognition and speech processing. More recently, neural network models started to be applied also to textual natural language signals, again with very promising results. This tutorial surveys neural network models from the perspective of natural language processing research, in an attempt to bring natural-language researchers up to speed with the neural techniques. The tutorial covers input encoding for natural language tasks, feed-forward networks, convolutional networks, recurrent networks and recursive networks, as well as the computation graph abstraction for automatic gradient computation.

RNN Basics

RNN Motivation

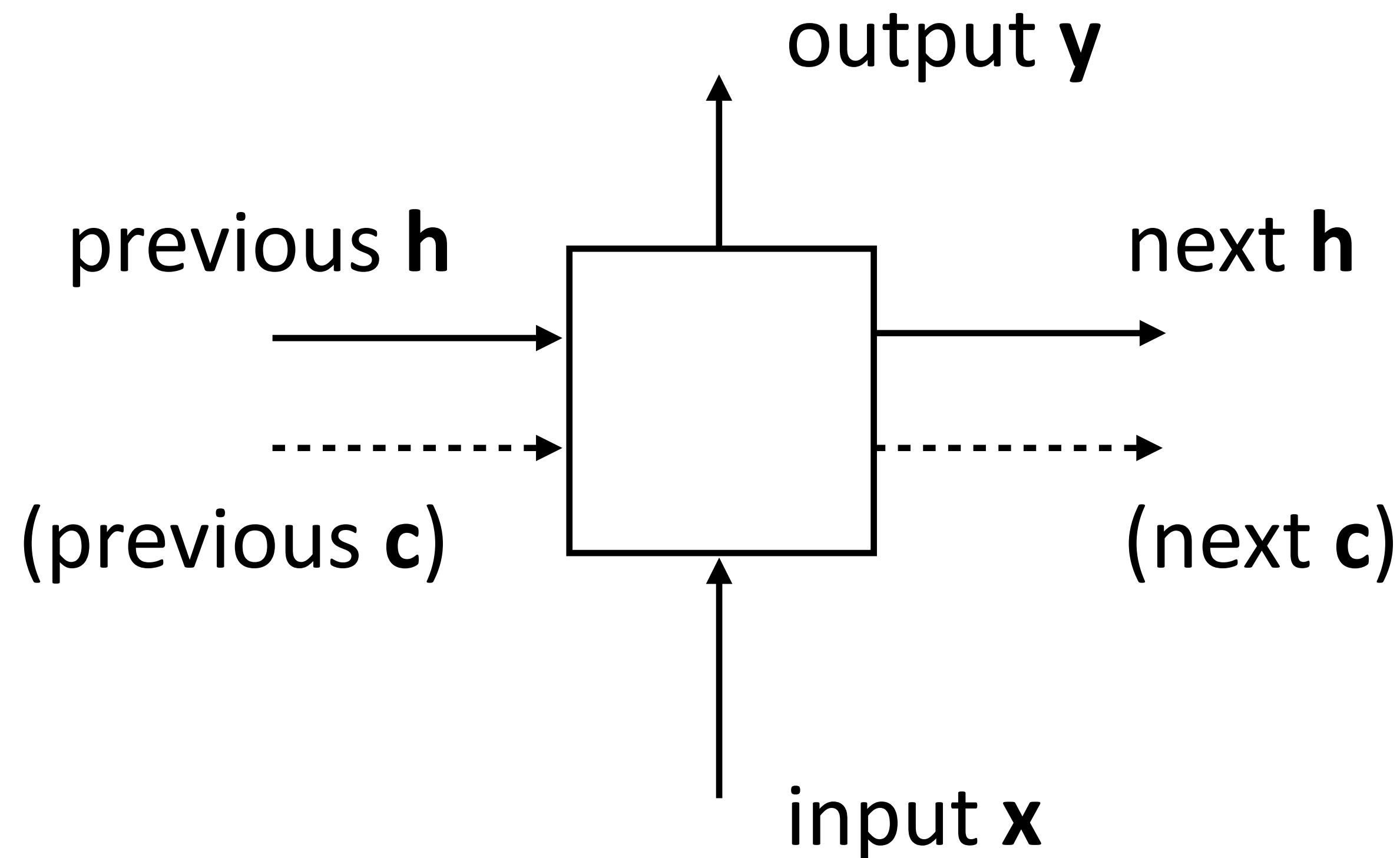
- ▶ Feedforward NNs isn't the best to handle sentences with variable length, words with multiple senses, or same words appear at different positions



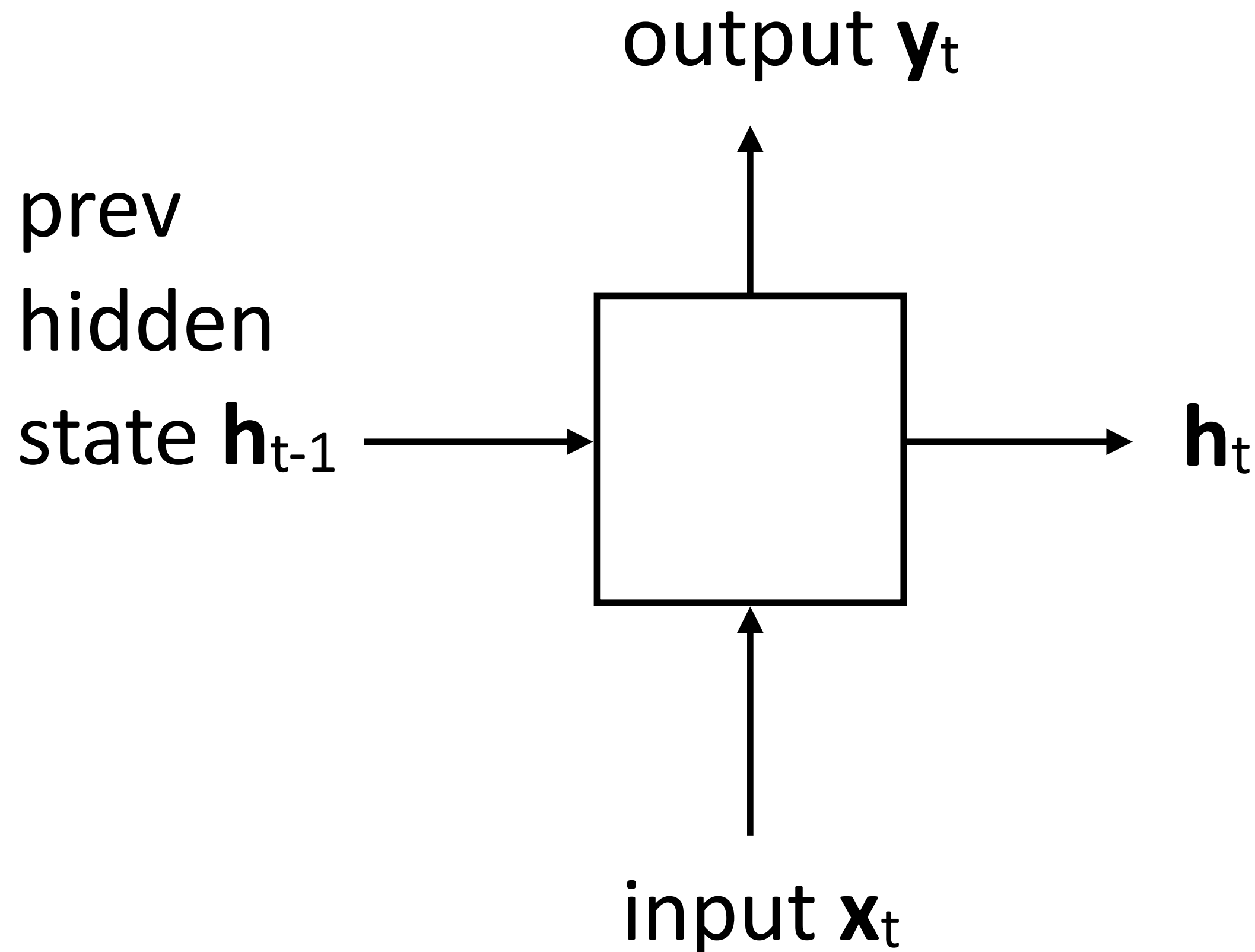
- ▶ These don't look related (*great* is in two different orthogonal subspaces)
- ▶ Instead, we need to:
 - 1) Process each word in a uniform way
 - 2) ...while still exploiting the context that that token occurs in

RNN Abstraction

- ▶ Cell that takes some input $\mathbf{x}$, has some hidden state $\mathbf{h}$, and updates that hidden state and produces output $\mathbf{y}$ (all vector-valued)



Elman Networks



$$\mathbf{h}_t = \tanh(W\mathbf{x}_t + V\mathbf{h}_{t-1} + \mathbf{b}_h)$$

- Updates hidden state based on input and previous hidden state

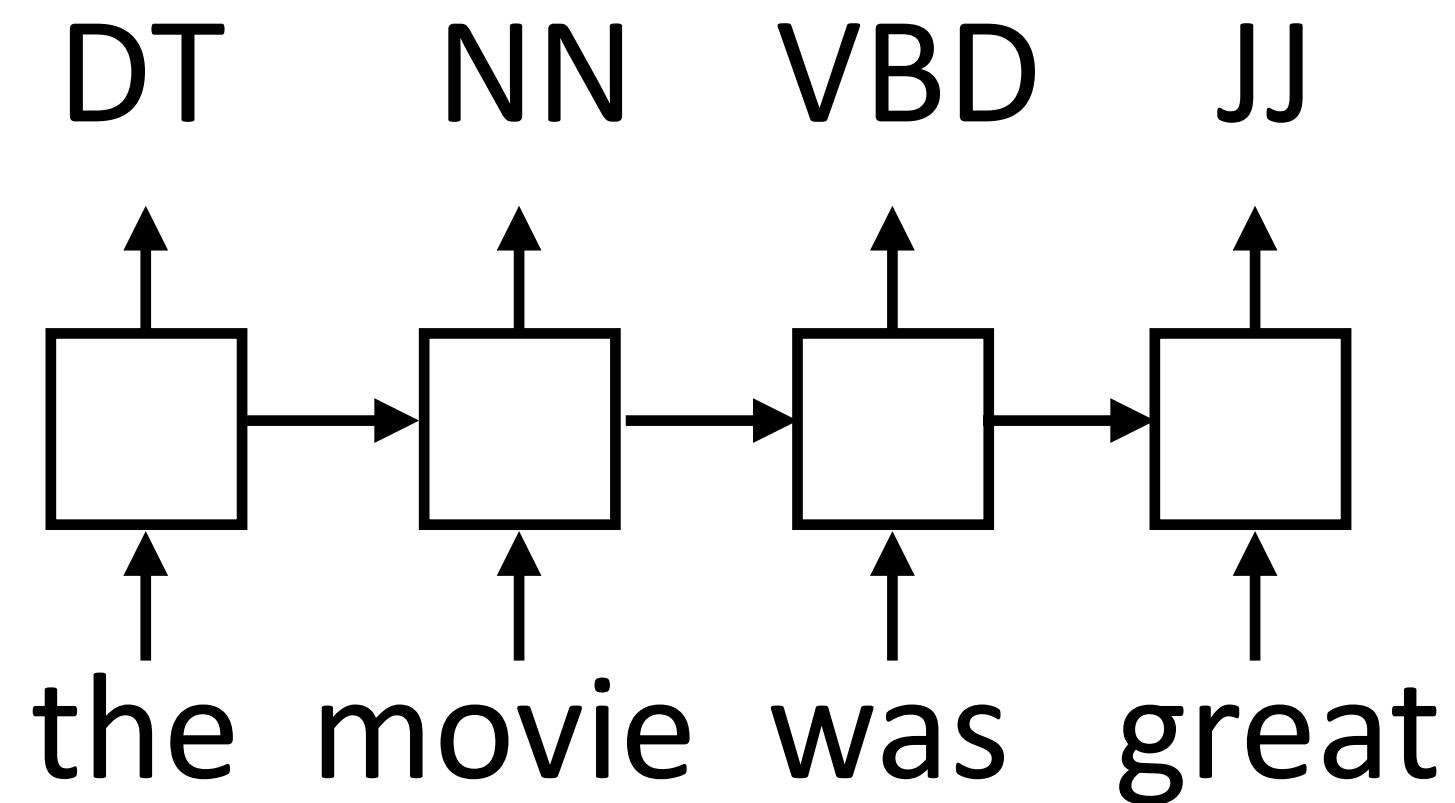
$$\mathbf{y}_t = \tanh(U\mathbf{h}_t + \mathbf{b}_y)$$

- Computes output from hidden state

- Long history! (invented in the late 1980s)

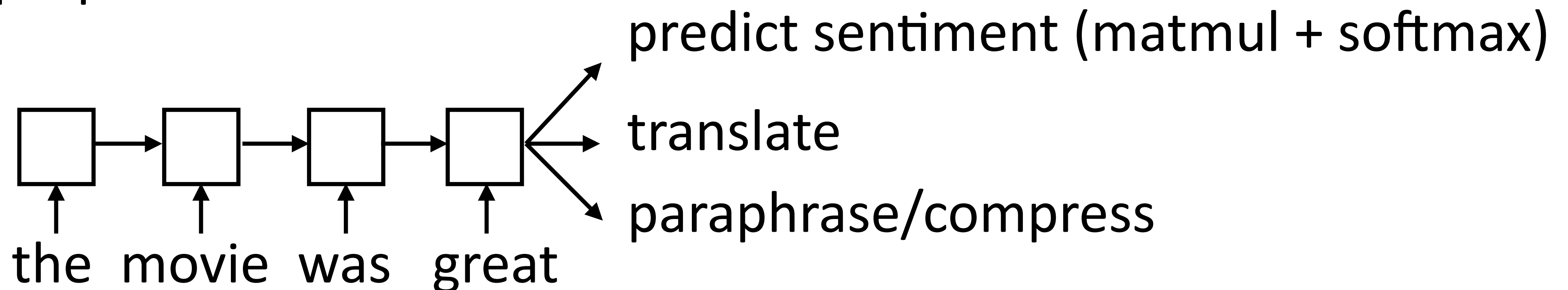
RNN Uses

- ▶ Transducer: make some prediction for each element in a sequence

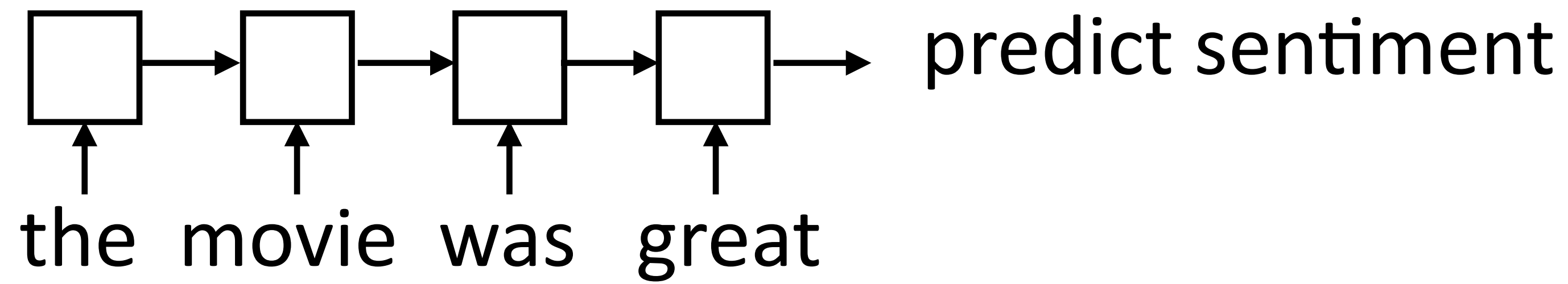


output $\mathbf{y}$ = score for each tag, then softmax

- ▶ Encoder: encode a sequence into a fixed-sized vector and use that for some purpose



RNN Intuition

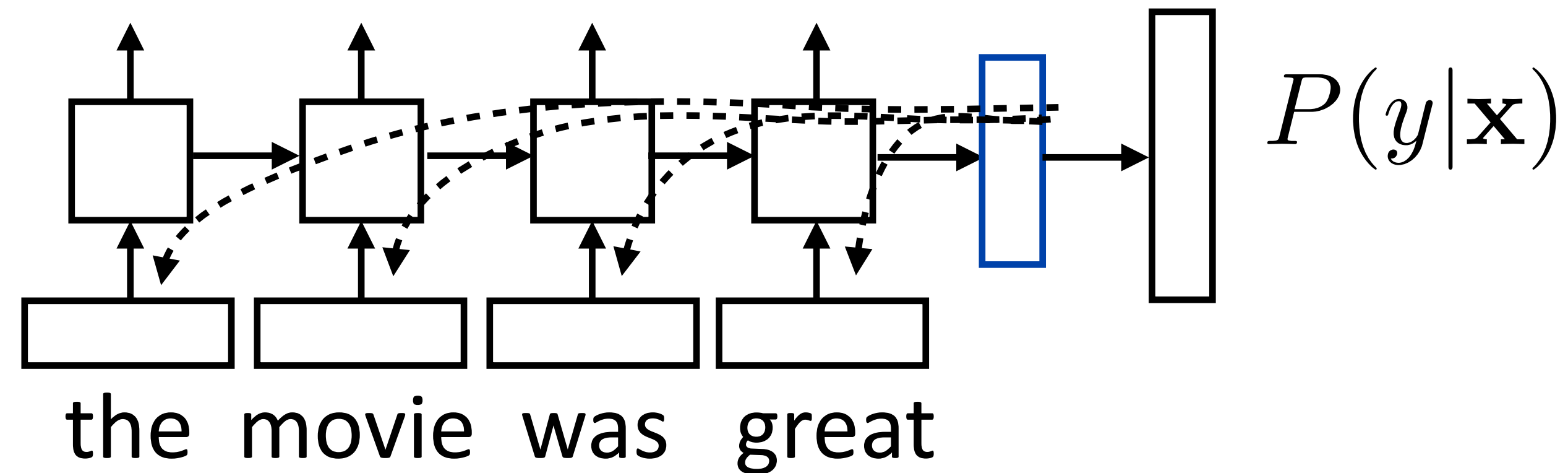


- ▶ RNN potentially needs to learn how to “remember” information for a long time!

it was my **favorite** movie of 2016, though it wasn't without **problems** -> +

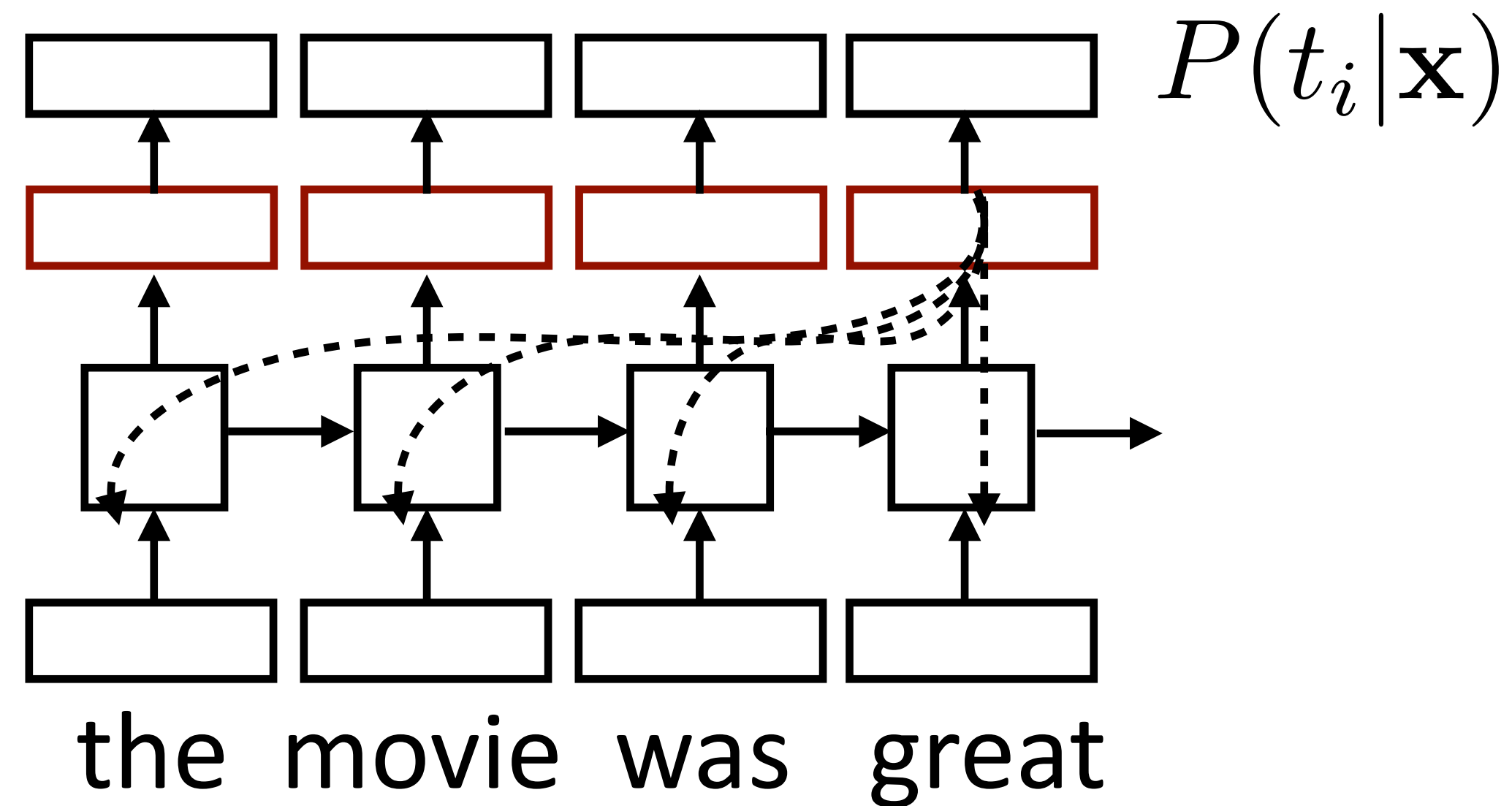
- ▶ “Correct” parameter update is to do a better job of remembering the sentiment of *favorite*

Training RNNs



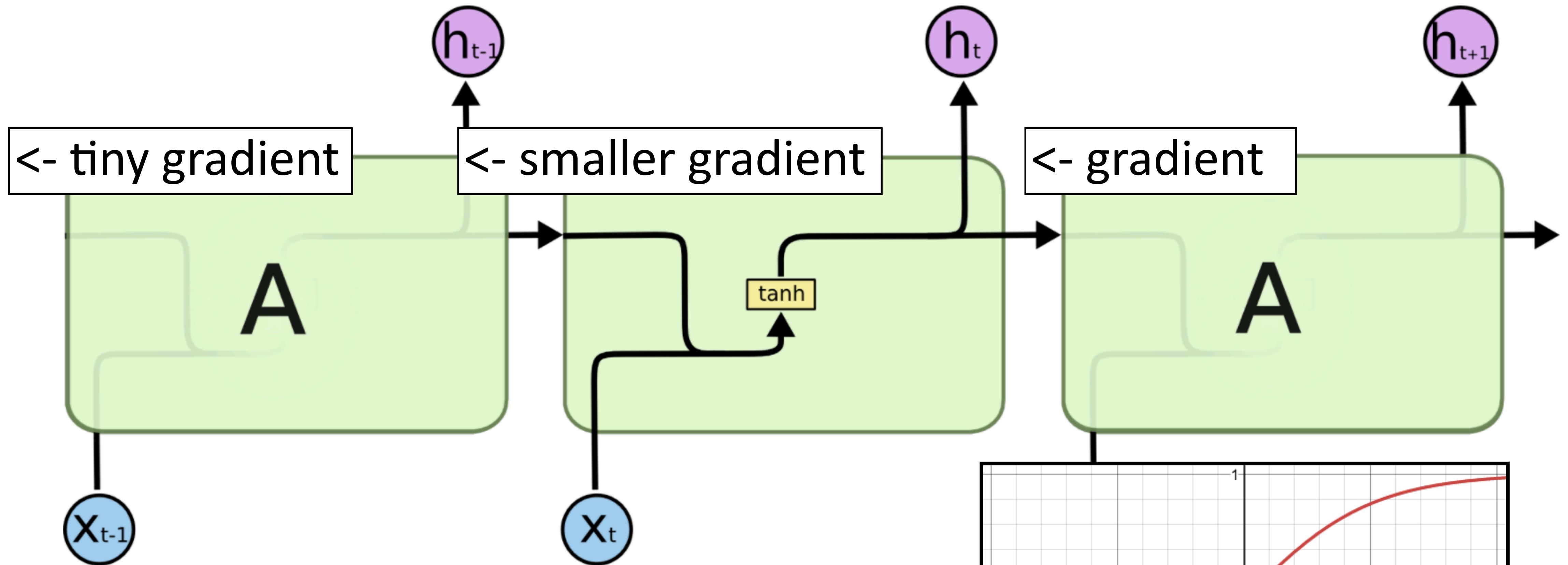
- ▶ Example: sentiment analysis
- ▶ Loss = negative log likelihood of probability of gold label (softmax or use SVM or other loss)
- ▶ “Backpropagation through time”: build the network as one big computation graph, some parameters are shared

Training RNNs

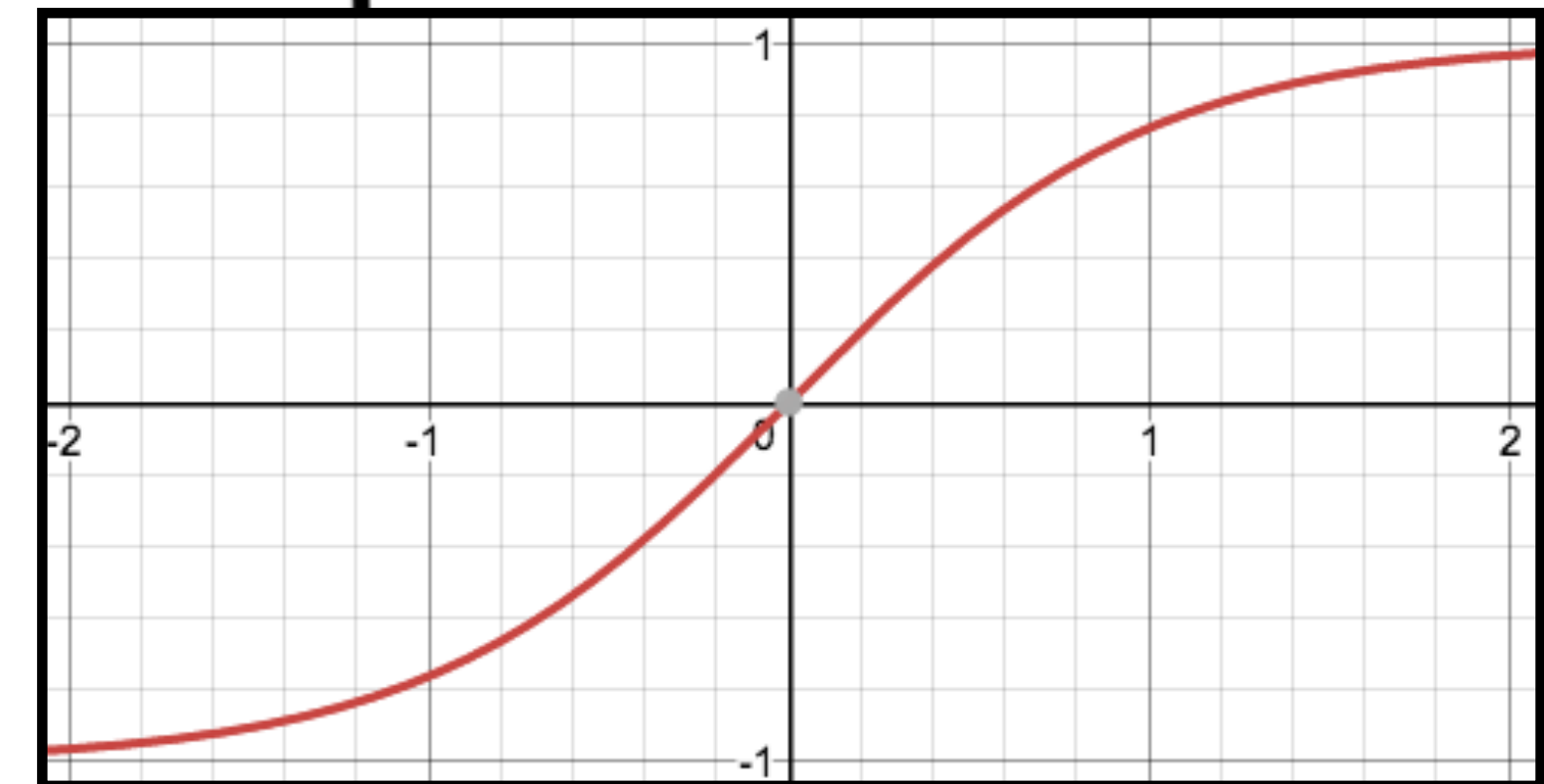


- ▶ Example: POS tagging, language modeling (predict next word given context)
- ▶ Loss = negative log likelihood of probability of gold predictions, summed over the tags
- ▶ Loss terms filter back through network

Vanishing Gradient



- ▶ Gradient diminishes going through tanh; if not in $[-2, 2]$, gradient is almost 0



LSTMs/GRUs

A Bit of History

- ▶ Long Short-term Memory (Hochreiter & Schmidhuber, 1997)

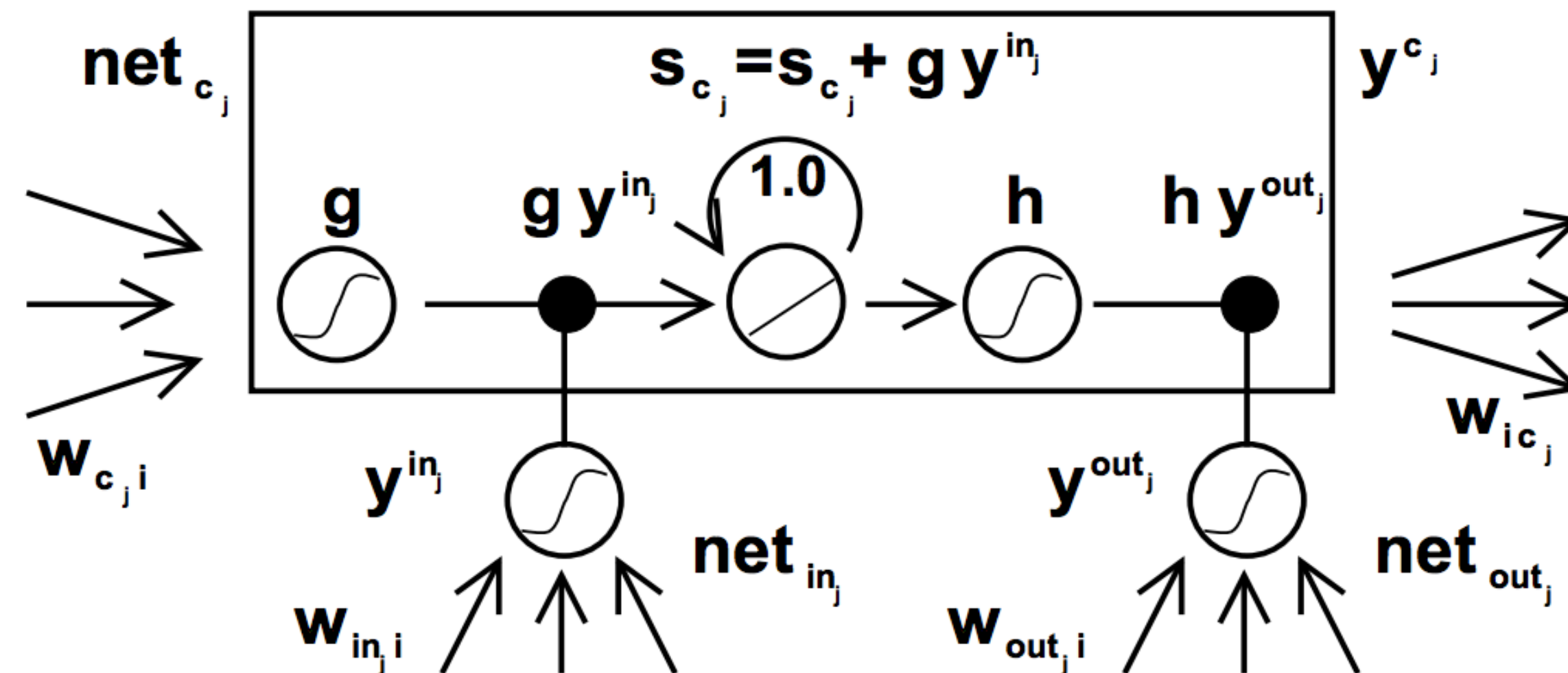


Figure 1: Architecture of memory cell c_j (the box) and its gate units in_j, out_j . The self-recurrent connection (with weight 1.0) indicates feedback with a delay of 1 time step. It builds the basis of the “constant error carrousel” CEC. The gate units open and close access to CEC. See text and appendix A.1 for details.

Gated Connections

- Designed to fix “vanishing gradient” problem using *gates*

$$\mathbf{h}_t = \mathbf{h}_{t-1} \odot \mathbf{f} + \text{func}(\mathbf{x}_t)$$

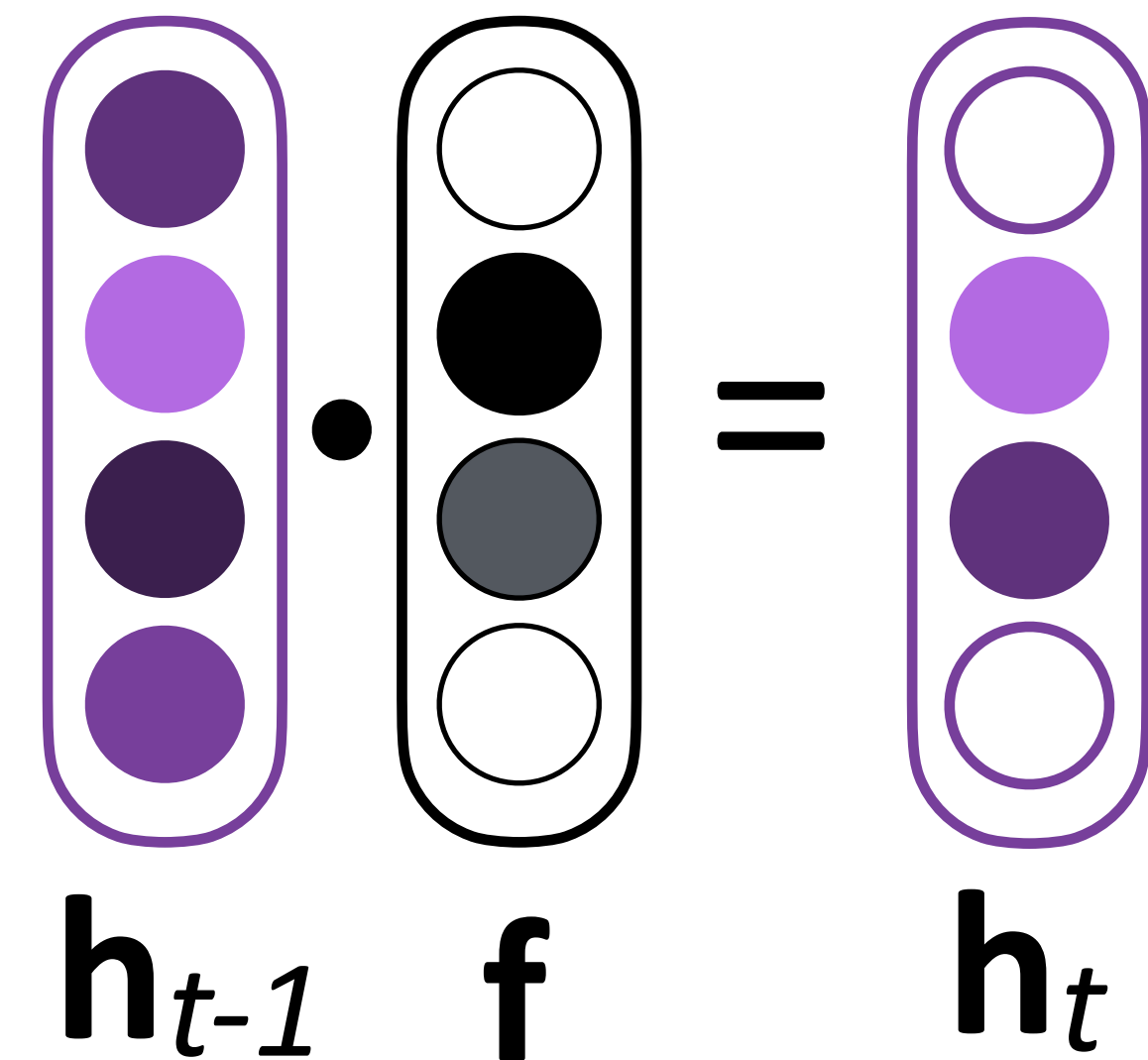
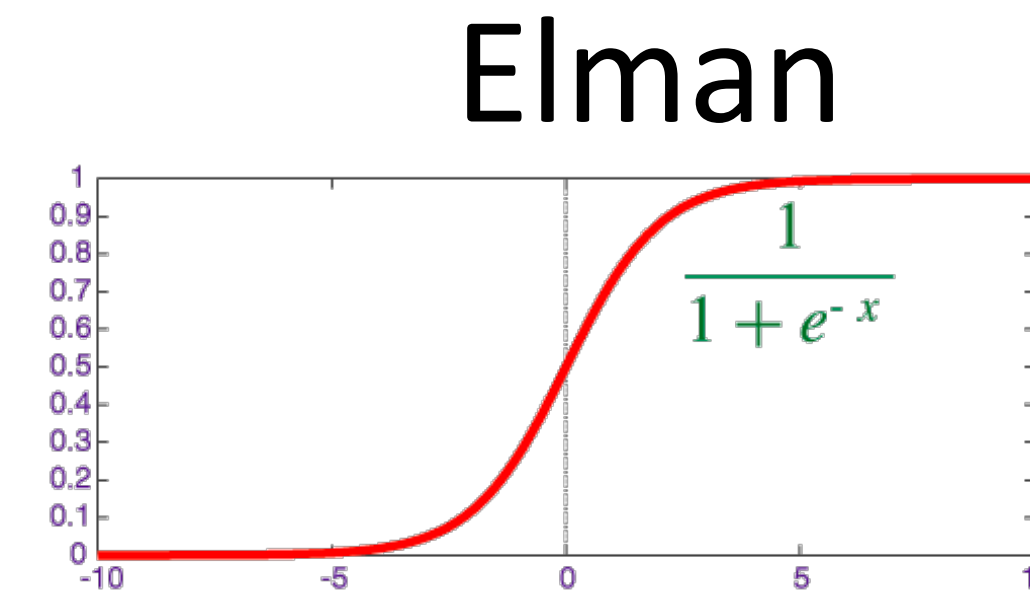
gated

$$\mathbf{h}_t = \tanh(W\mathbf{x}_t + V\mathbf{h}_{t-1} + \mathbf{b}_h)$$

- Vector-valued “forget gate” $\mathbf{f}$ computed based on input and previous hidden state

$$\mathbf{f} = \sigma(W^{xf}\mathbf{x}_t + W^{hf}\mathbf{h}_{t-1})$$

- Sigmoid: elements of $\mathbf{f}$ are in $(0, 1)$
- If $\mathbf{f} \approx \mathbf{1}$, we simply sum up a function of all inputs — gradient doesn’t vanish!



LSTMs

- ▶ Long short-term memory network: hidden state as a “short-term” memory

- ▶ “Cell” $\mathbf{c}$ in addition to hidden state $\mathbf{h}$

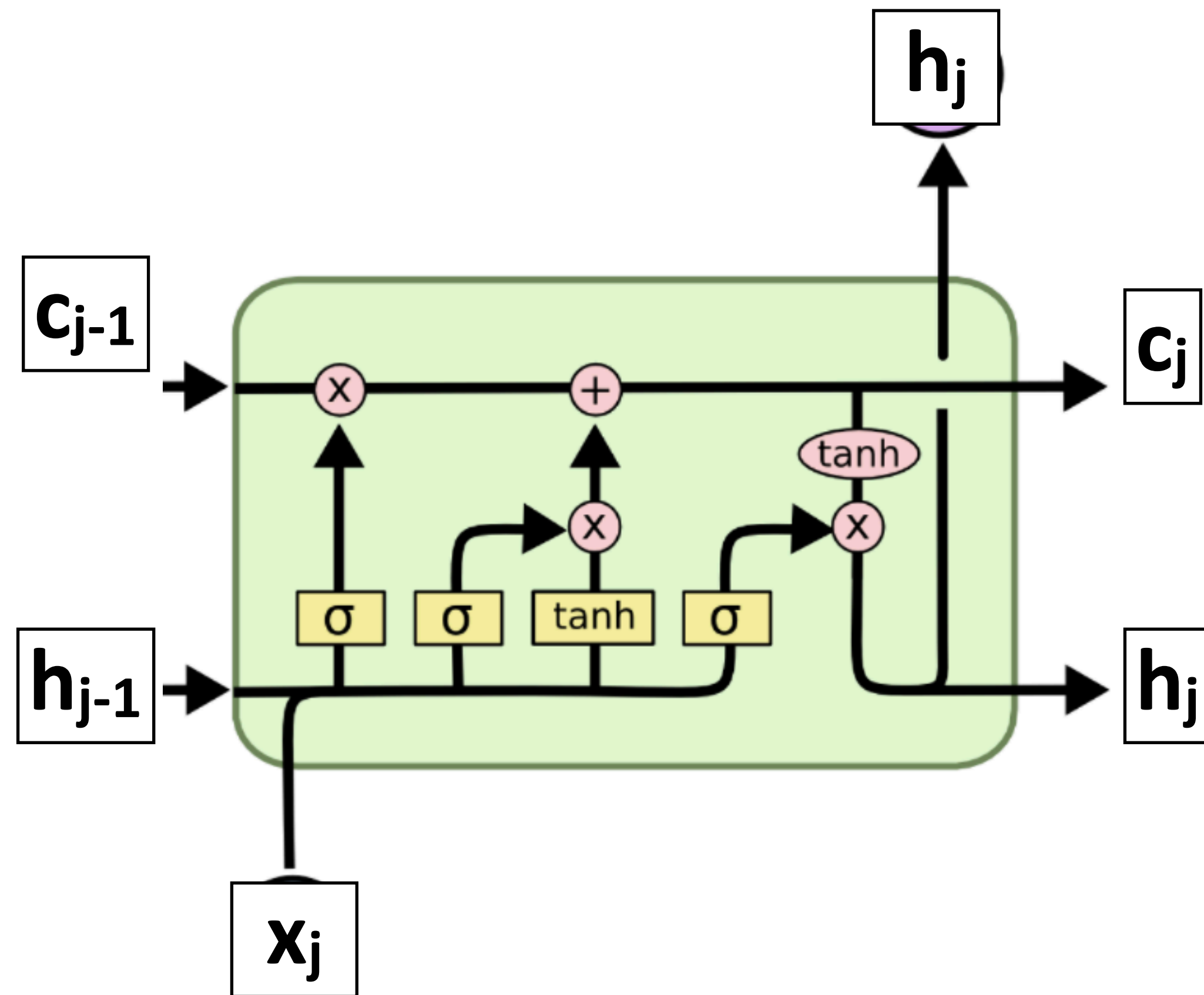
$$\mathbf{c}_t = \mathbf{c}_{t-1} \odot \mathbf{f} + \text{func}(\mathbf{x}_t, \mathbf{h}_{t-1})$$

- ▶ Vector-valued forget gate $\mathbf{f}$ depends on the $\mathbf{h}$ hidden state

$$\mathbf{f} = \sigma(W^{xf}\mathbf{x}_t + W^{hf}\mathbf{h}_{t-1})$$

- ▶ Basic communication flow: $\mathbf{x} \rightarrow \mathbf{c} \rightarrow \mathbf{h} \rightarrow \text{output}$, each step of this process is gated in addition to gates from previous timesteps

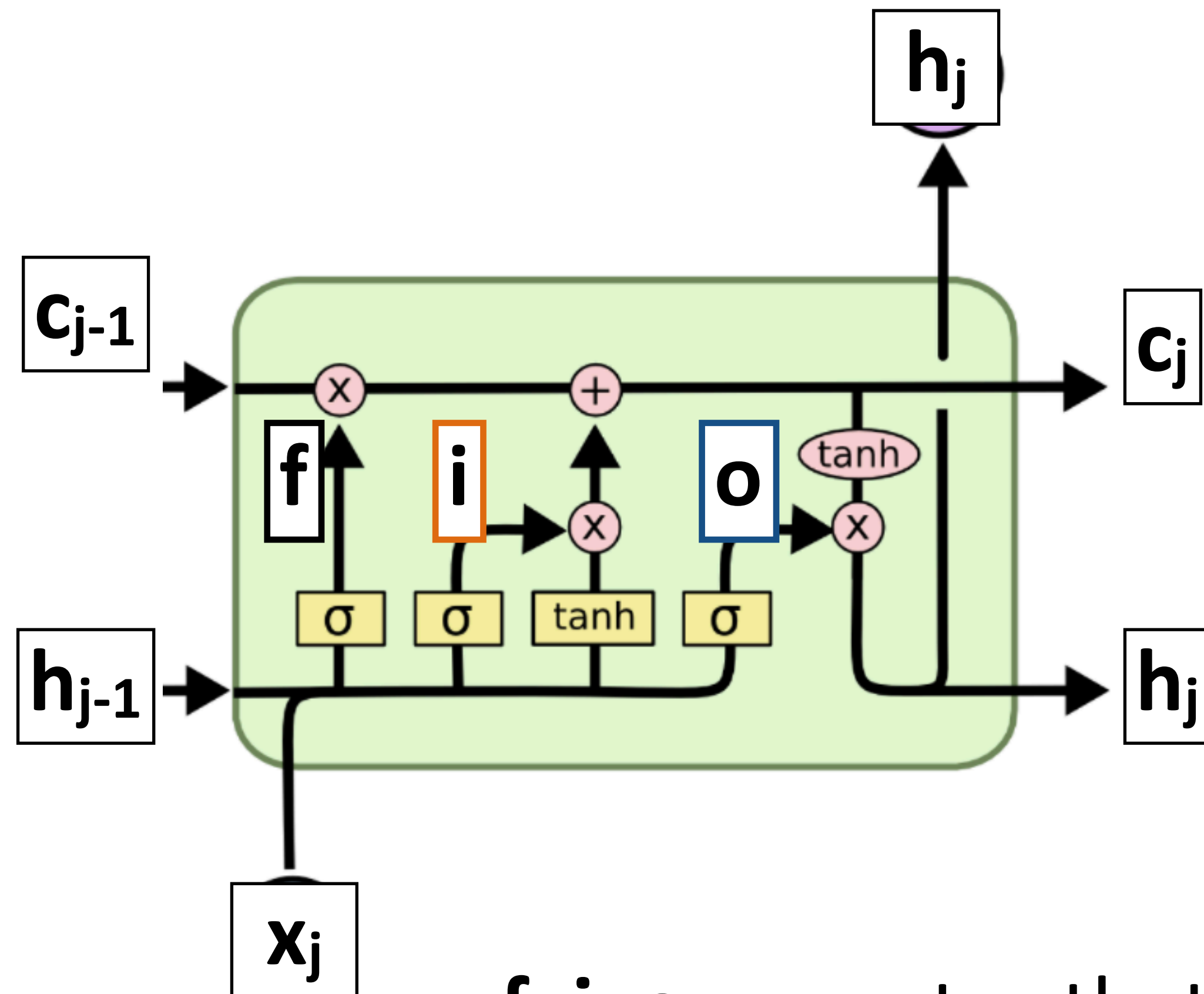
LSTMs



Hochreiter & Schmidhuber (1997)

<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

LSTMs



$$f = \sigma(x_j W^{xf} + h_{j-1} W^{hf})$$

$$i = \sigma(x_j W^{xi} + h_{j-1} W^{hi})$$

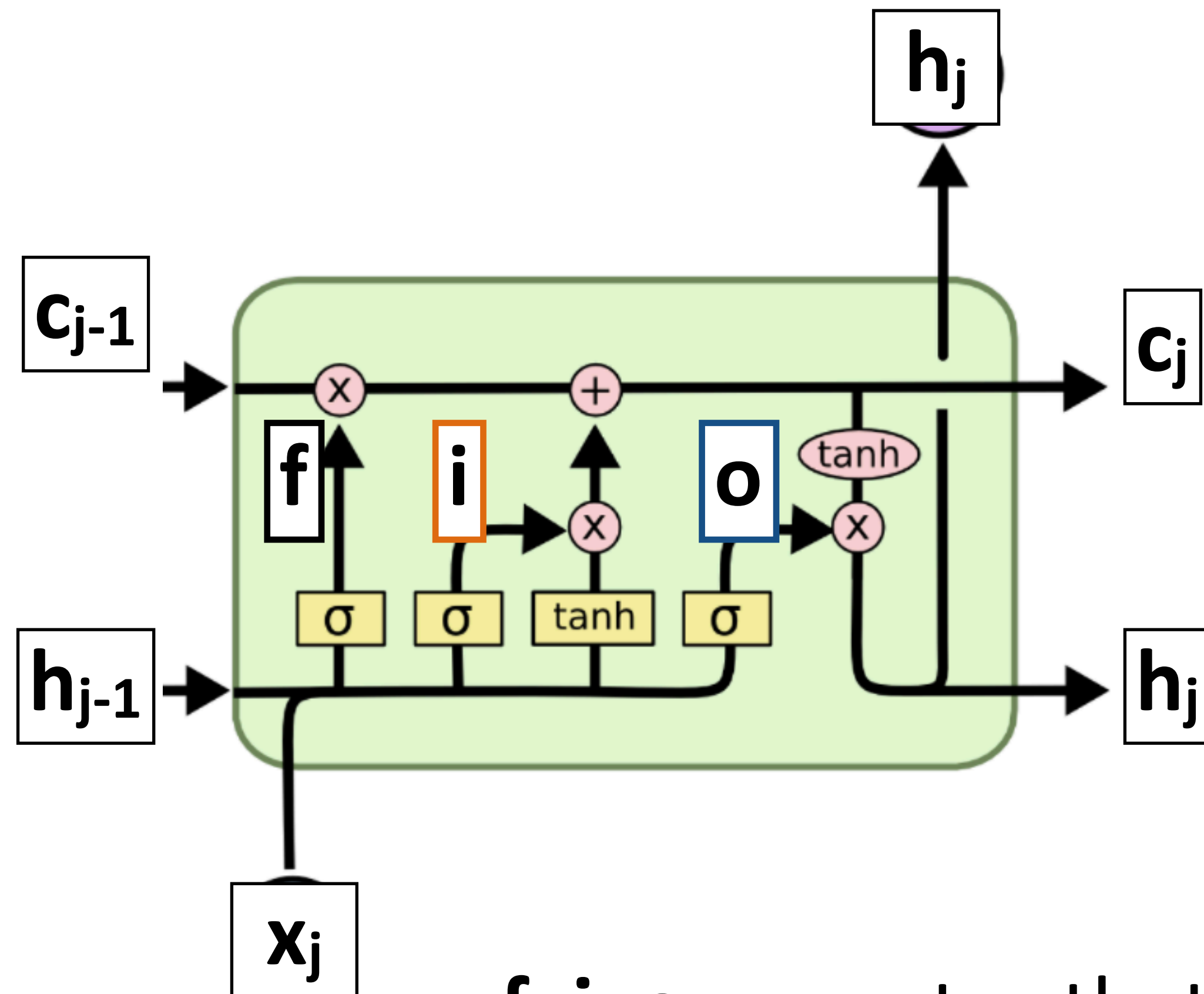
$$o = \sigma(x_j W^{xo} + h_{j-1} W^{ho})$$

- ▶ f , i , o are gates that control information flow

Hochreiter & Schmidhuber (1997)

<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

LSTMs



$$f = \sigma(x_j W^{xf} + h_{j-1} W^{hf})$$

$$g = \tanh(x_j W^{xg} + h_{j-1} W^{hg})$$

$$i = \sigma(x_j W^{xi} + h_{j-1} W^{hi})$$

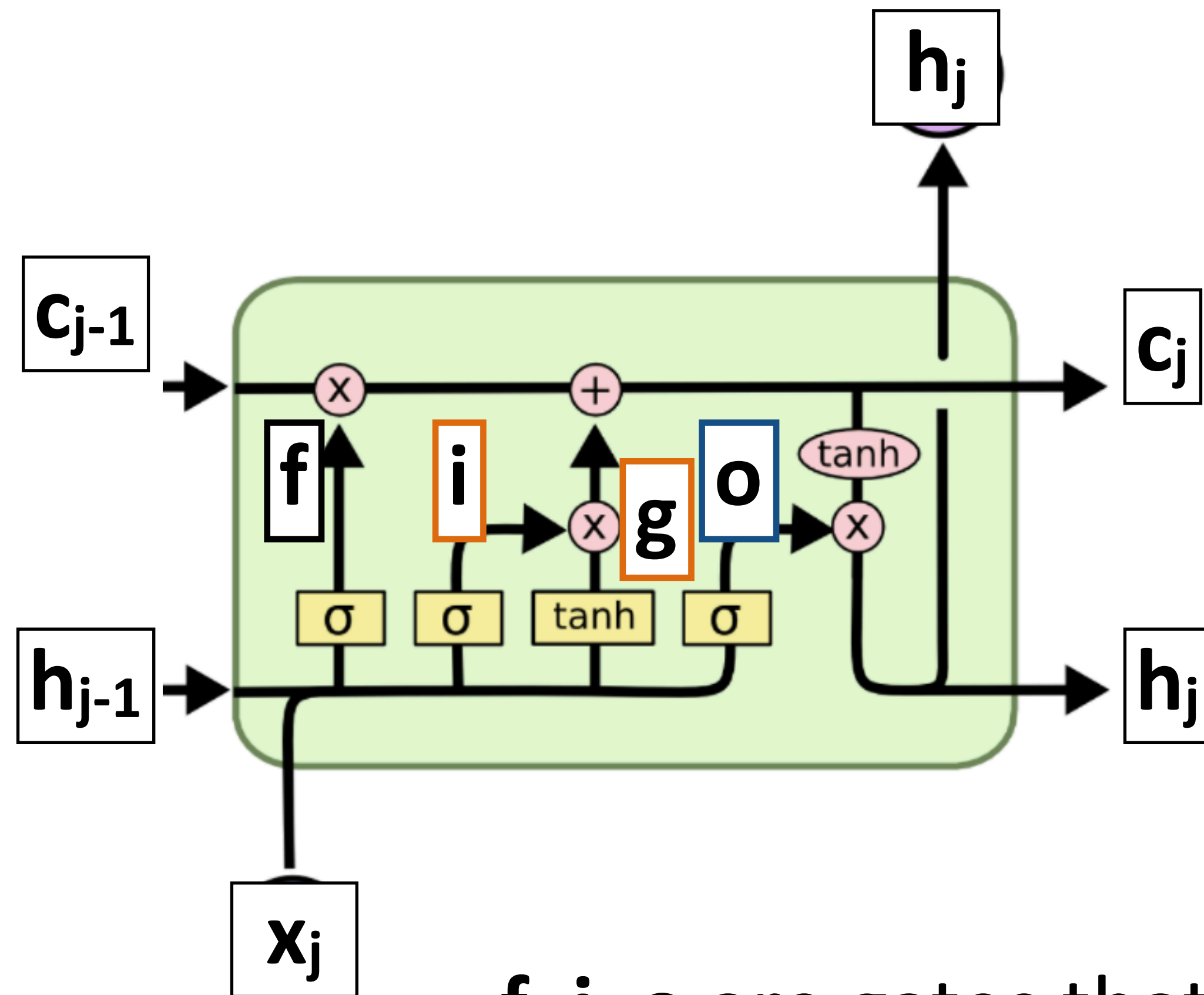
$$o = \sigma(x_j W^{xo} + h_{j-1} W^{ho})$$

- ▶ f , i , o are gates that control information flow
- ▶ g reflects the main computation of the cell

Hochreiter & Schmidhuber (1997)

<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

LSTMs



$$c_j = c_{j-1} \odot f + g \odot i$$

$$f = \sigma(x_j W^{xf} + h_{j-1} W^{hf})$$

$$g = \tanh(x_j W^{xg} + h_{j-1} W^{hg})$$

$$i = \sigma(x_j W^{xi} + h_{j-1} W^{hi})$$

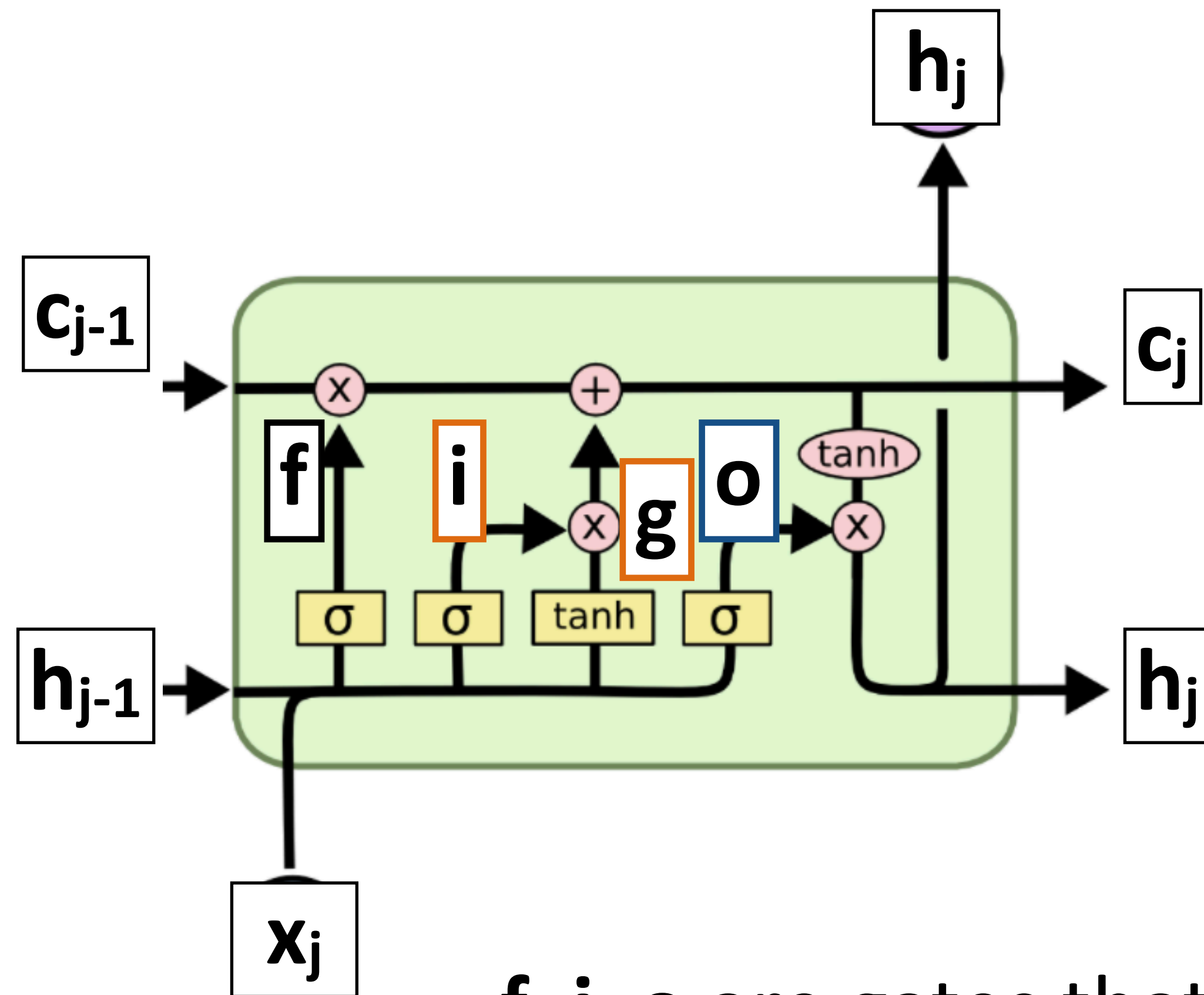
$$o = \sigma(x_j W^{xo} + h_{j-1} W^{ho})$$

- ▶ f , i , o are gates that control information flow
- ▶ g reflects the main computation of the cell

Hochreiter & Schmidhuber (1997)

<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

LSTMs



$$c_j = c_{j-1} \odot f + g \odot i$$

$$f = \sigma(x_j W^{xf} + h_{j-1} W^{hf})$$

$$g = \tanh(x_j W^{xg} + h_{j-1} W^{hg})$$

$$i = \sigma(x_j W^{xi} + h_{j-1} W^{hi})$$

$$h_j = \tanh(c_j) \odot o$$

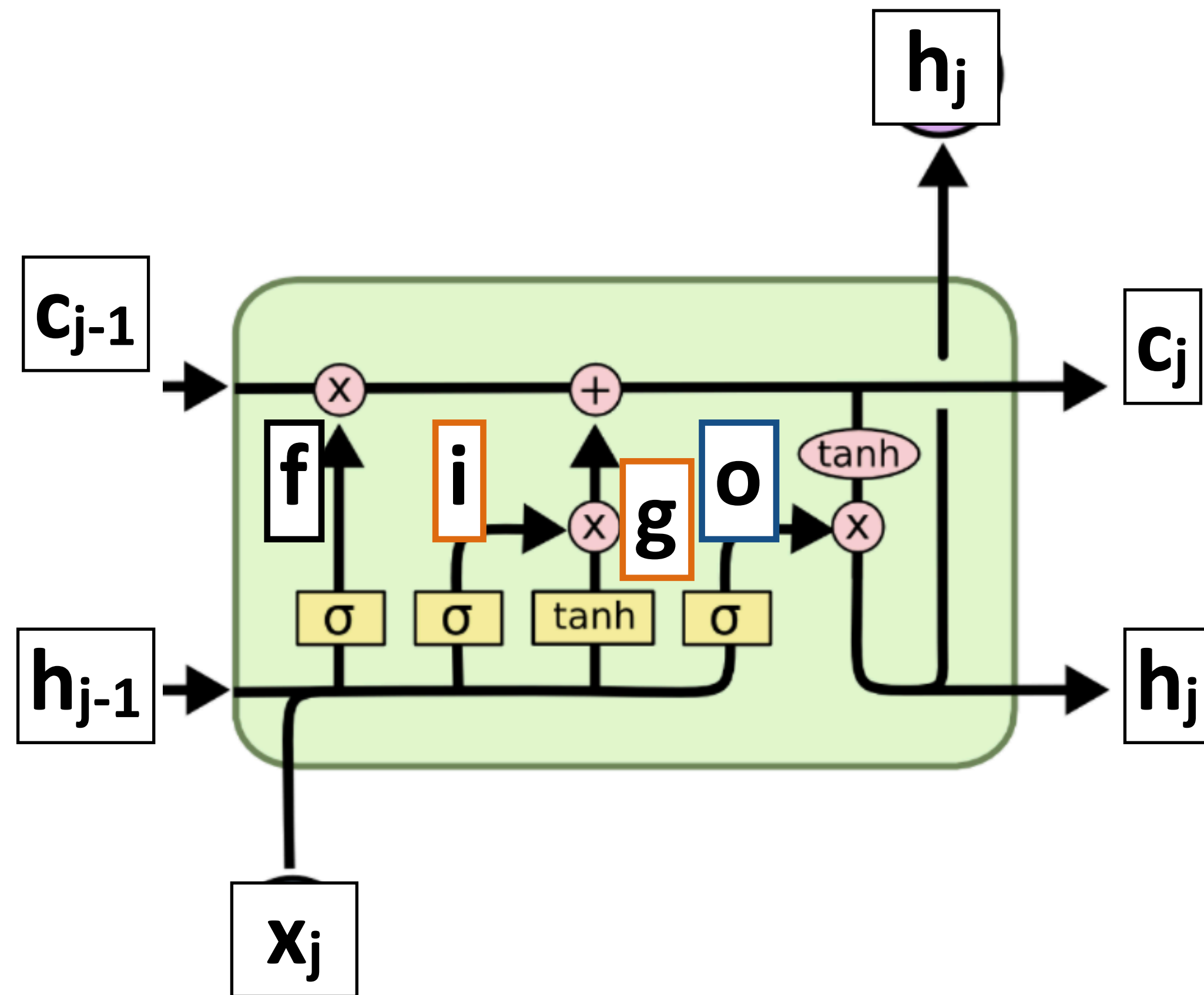
$$o = \sigma(x_j W^{xo} + h_{j-1} W^{ho})$$

- ▶ f , i , o are gates that control information flow
- ▶ g reflects the main computation of the cell

Hochreiter & Schmidhuber (1997)

<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

LSTMs



$$c_j = c_{j-1} \odot f + g \odot i$$

$$f = \sigma(x_j W^{xf} + h_{j-1} W^{hf})$$

$$g = \tanh(x_j W^{xg} + h_{j-1} W^{hg})$$

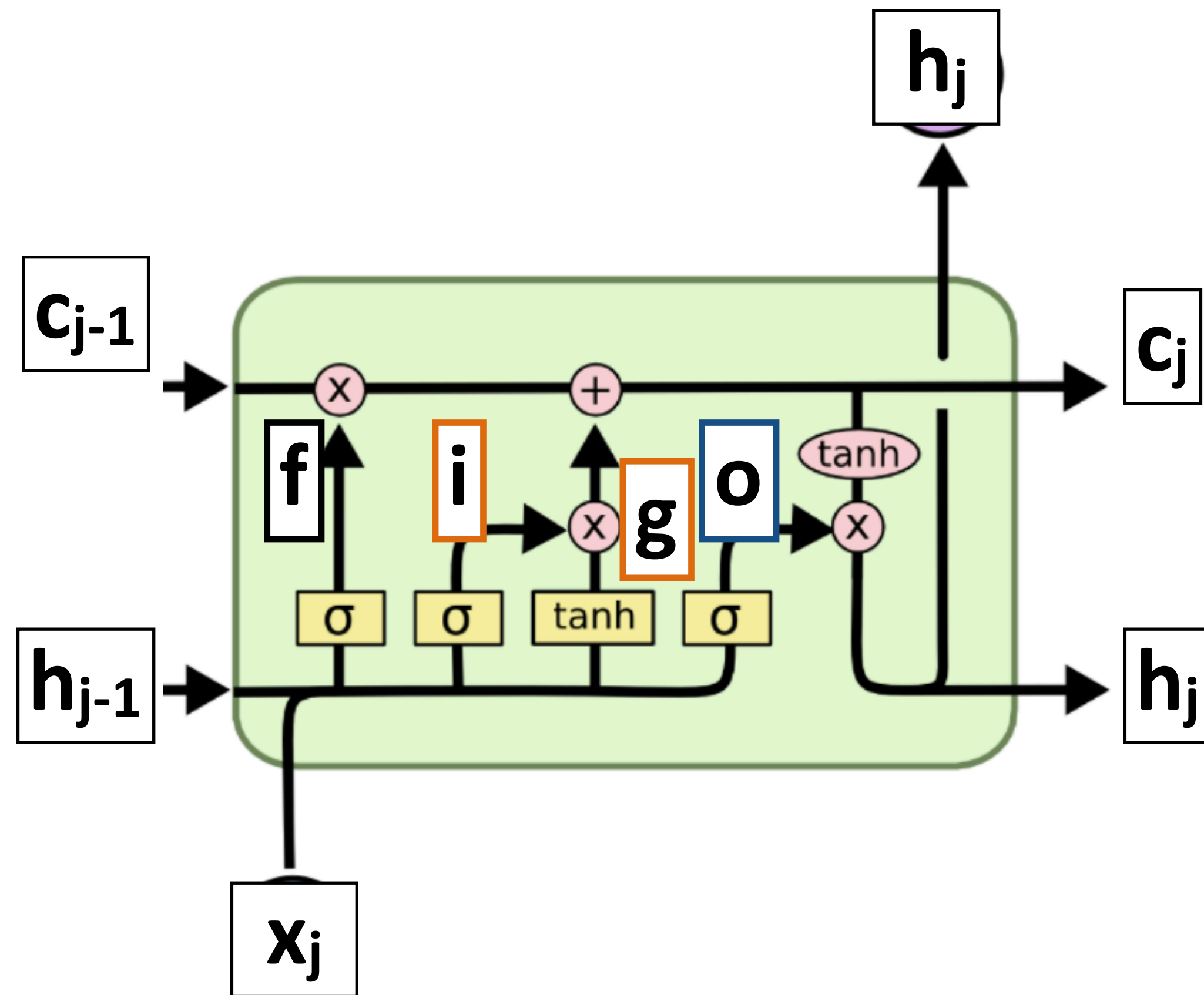
$$i = \sigma(x_j W^{xi} + h_{j-1} W^{hi})$$

$$h_j = \tanh(c_j) \odot o$$

$$o = \sigma(x_j W^{xo} + h_{j-1} W^{ho})$$

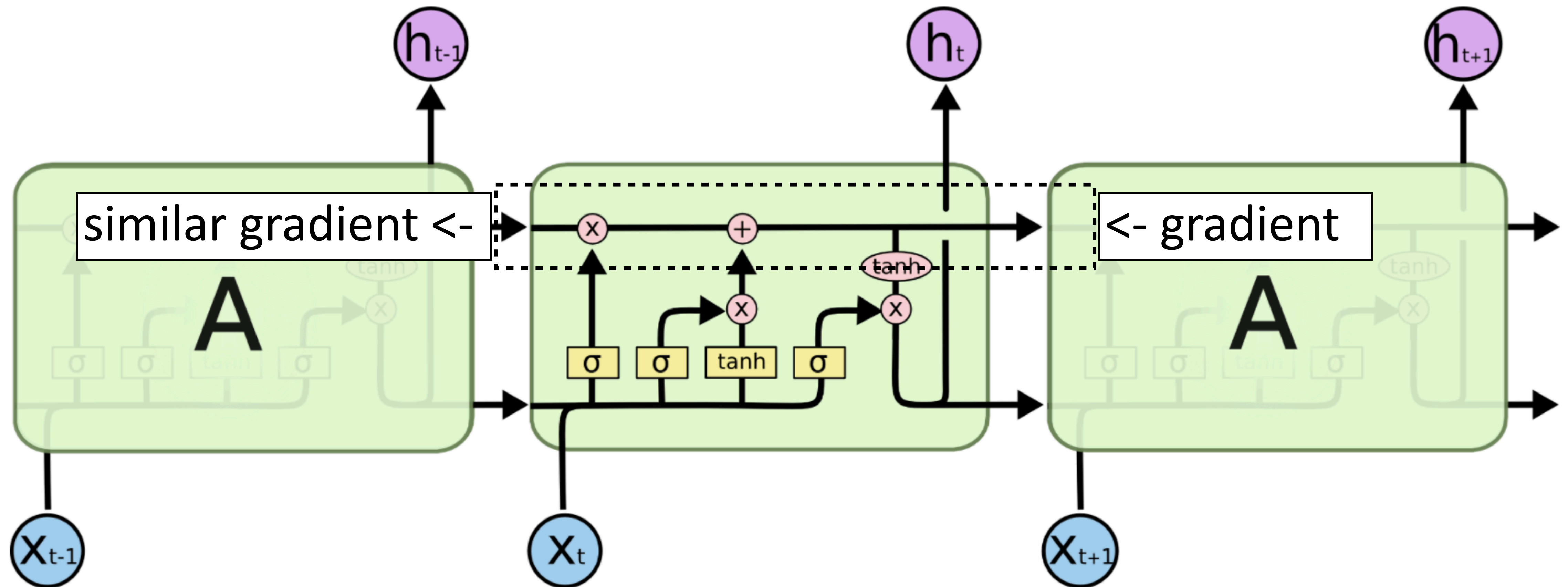
- ▶ Can we ignore the old value of c for this timestep?
- ▶ Can an LSTM sum up its inputs x ?
- ▶ Can we ignore a particular input x ?

LSTMs



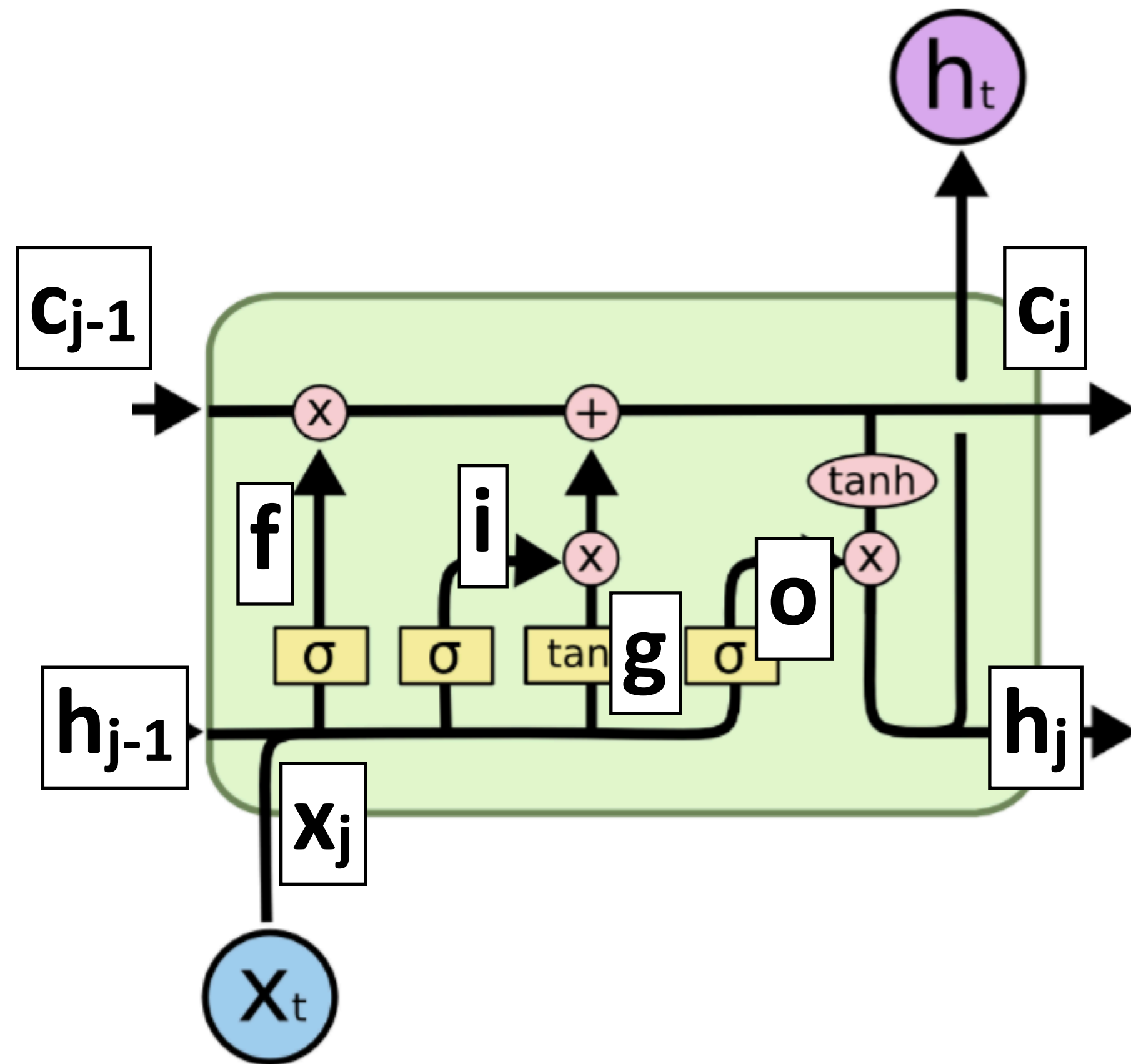
- ▶ Ignoring recurrent state entirely:
 - ▶ Lets us get feedforward layer over token
- ▶ Ignoring input:
 - ▶ Lets us discard stopwords
- ▶ Summing inputs:
 - ▶ Lets us compute a bag-of-words representation

LSTMs



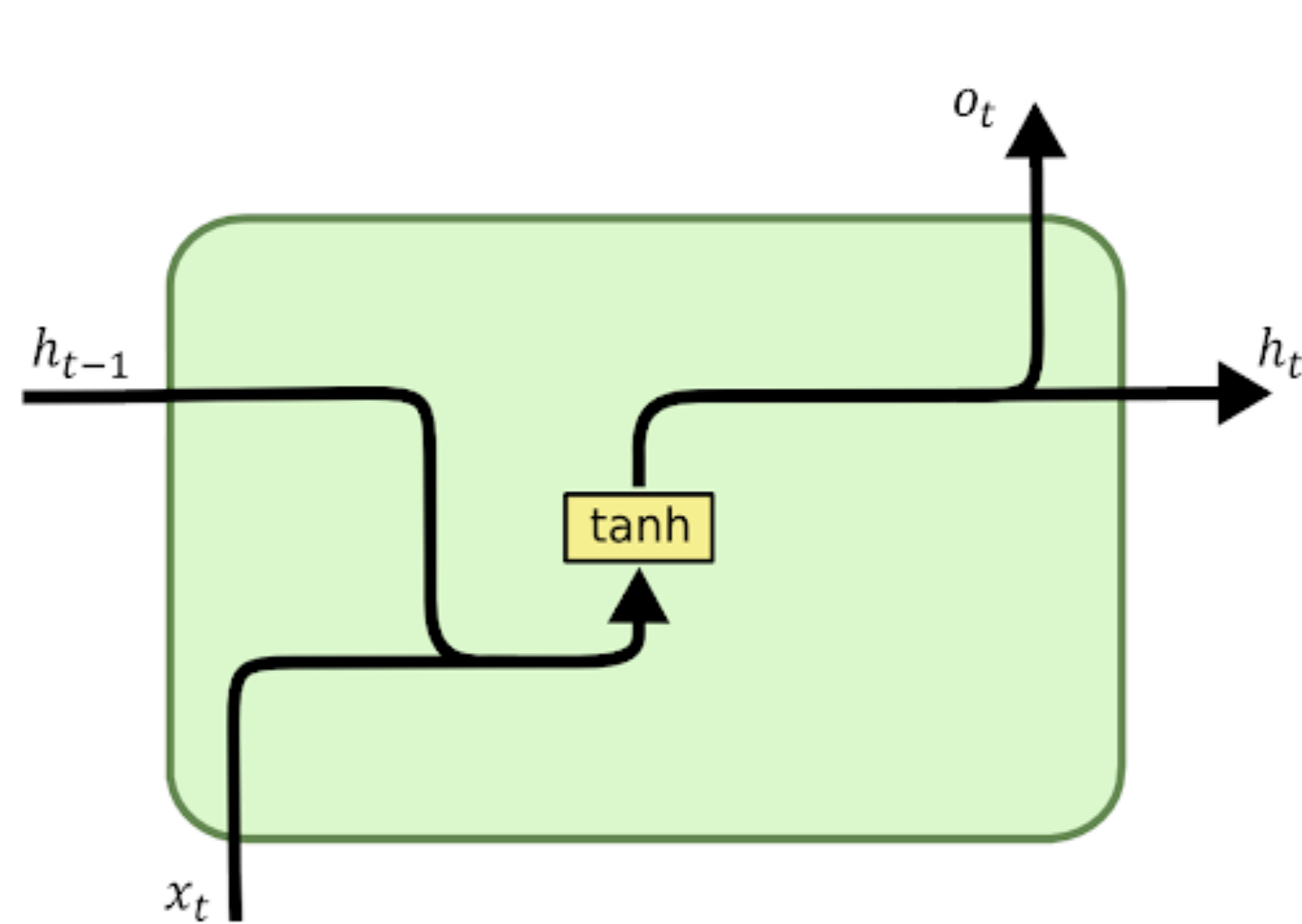
- ▶ Gradient still diminishes, but in a controlled way and generally by less — usually initialize forget gate = 1 to remember everything to start

Understanding LSTM Parameters

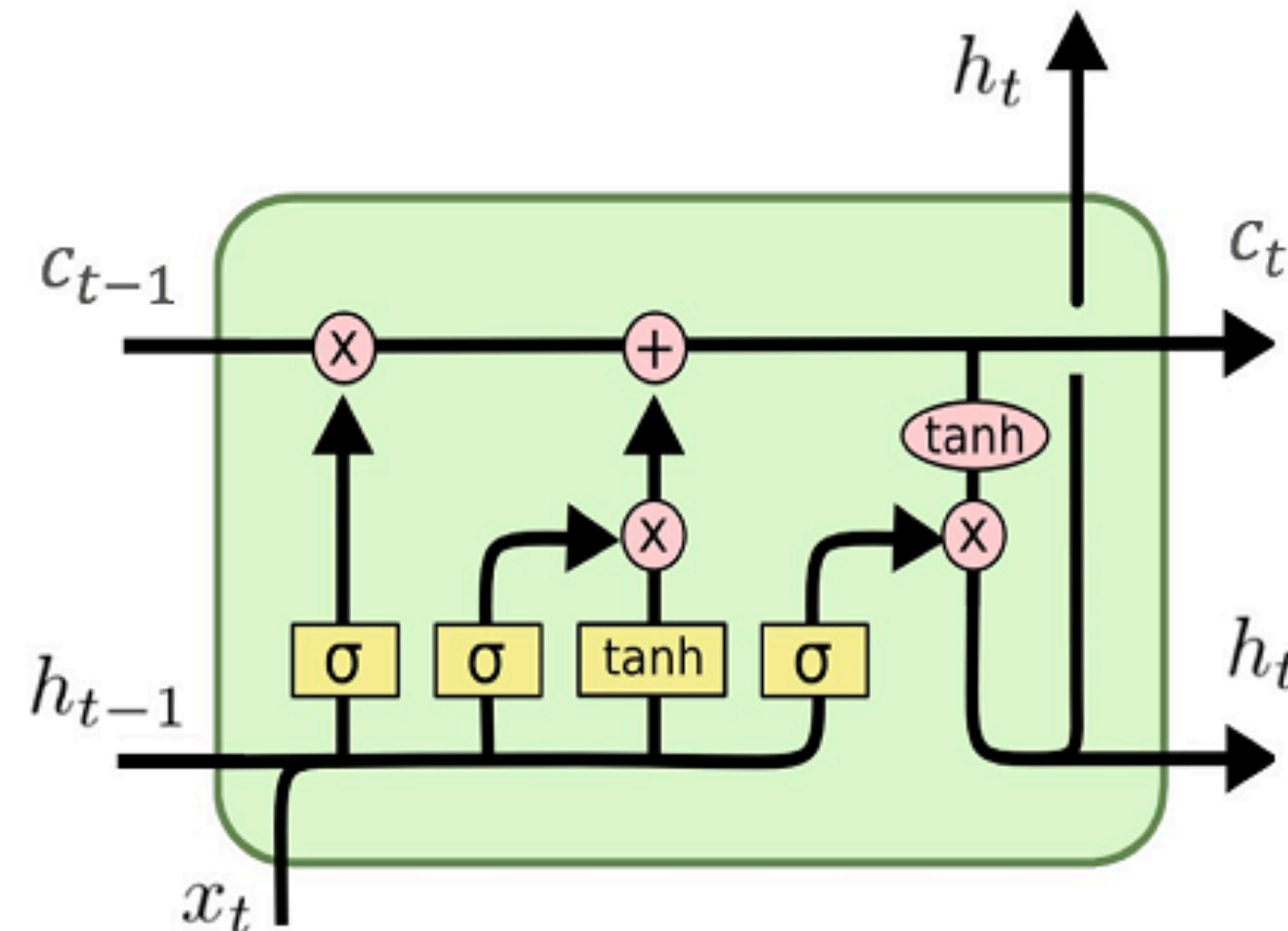


- ▶ Initialize hidden layer randomly
- ▶ Need to learn how the gates work: what do we forget/remember?
- ▶ g uses an arbitrary nonlinearity, this is the “layer” of the cell

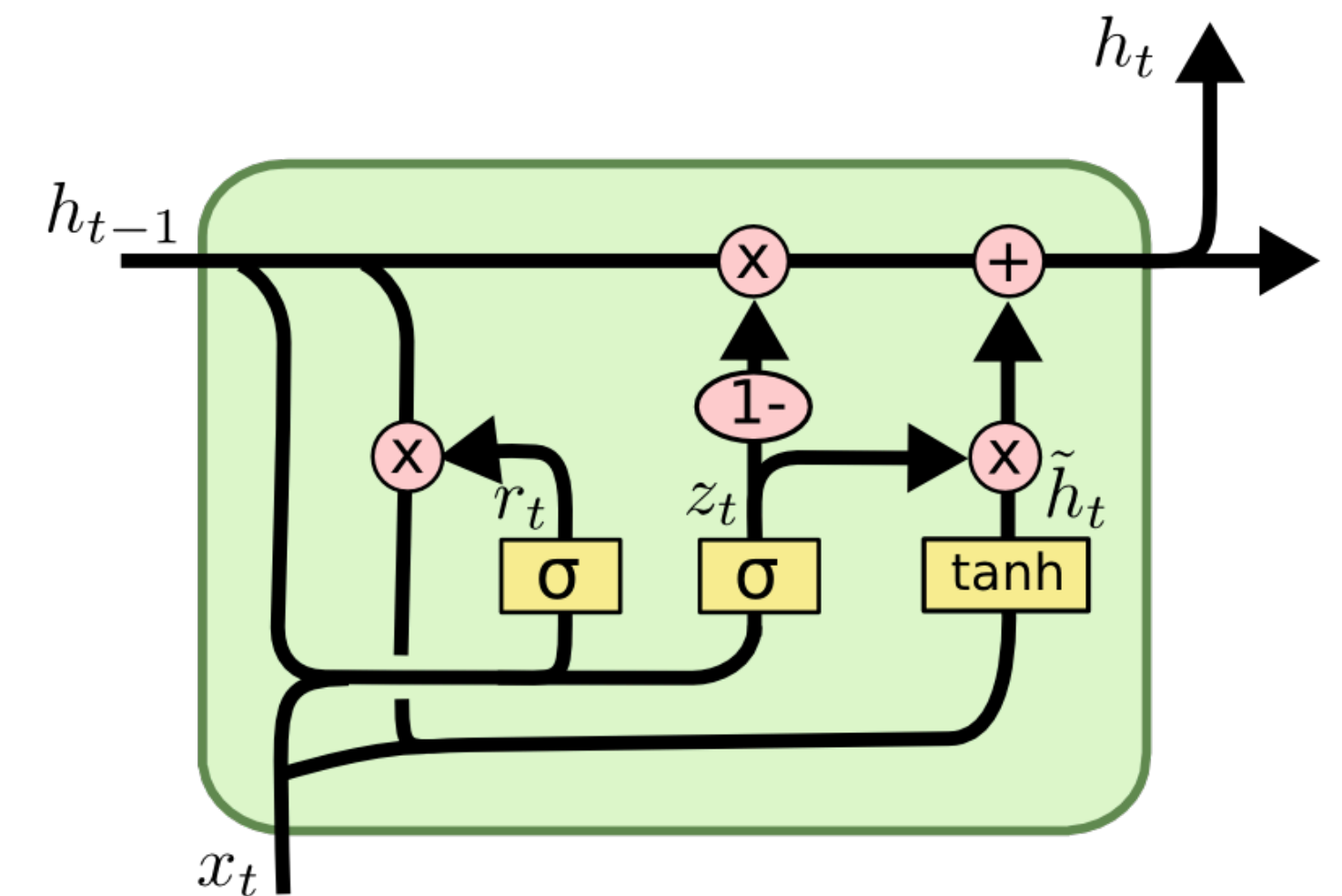
Standard RNN vs. LSTM vs. GRU



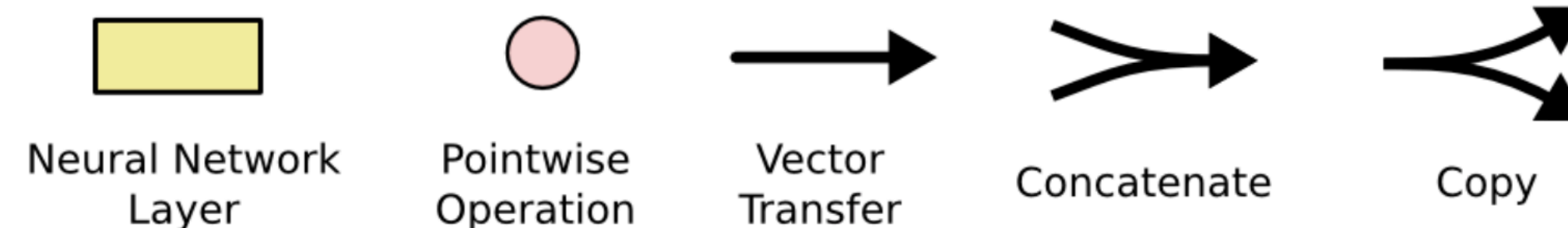
Standard RNN
(with a single layer)



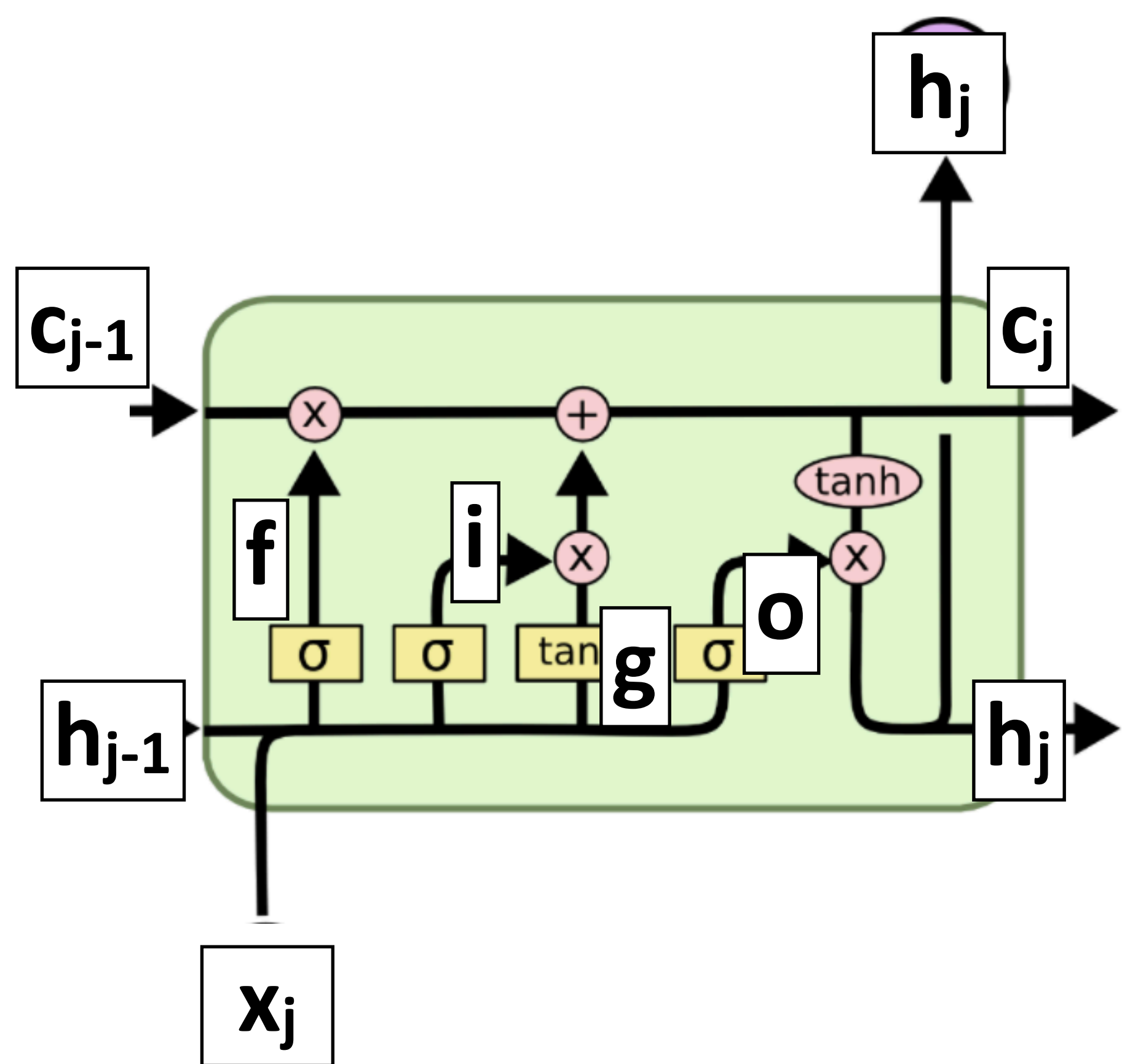
LSTM
(Long Short-term Memory)



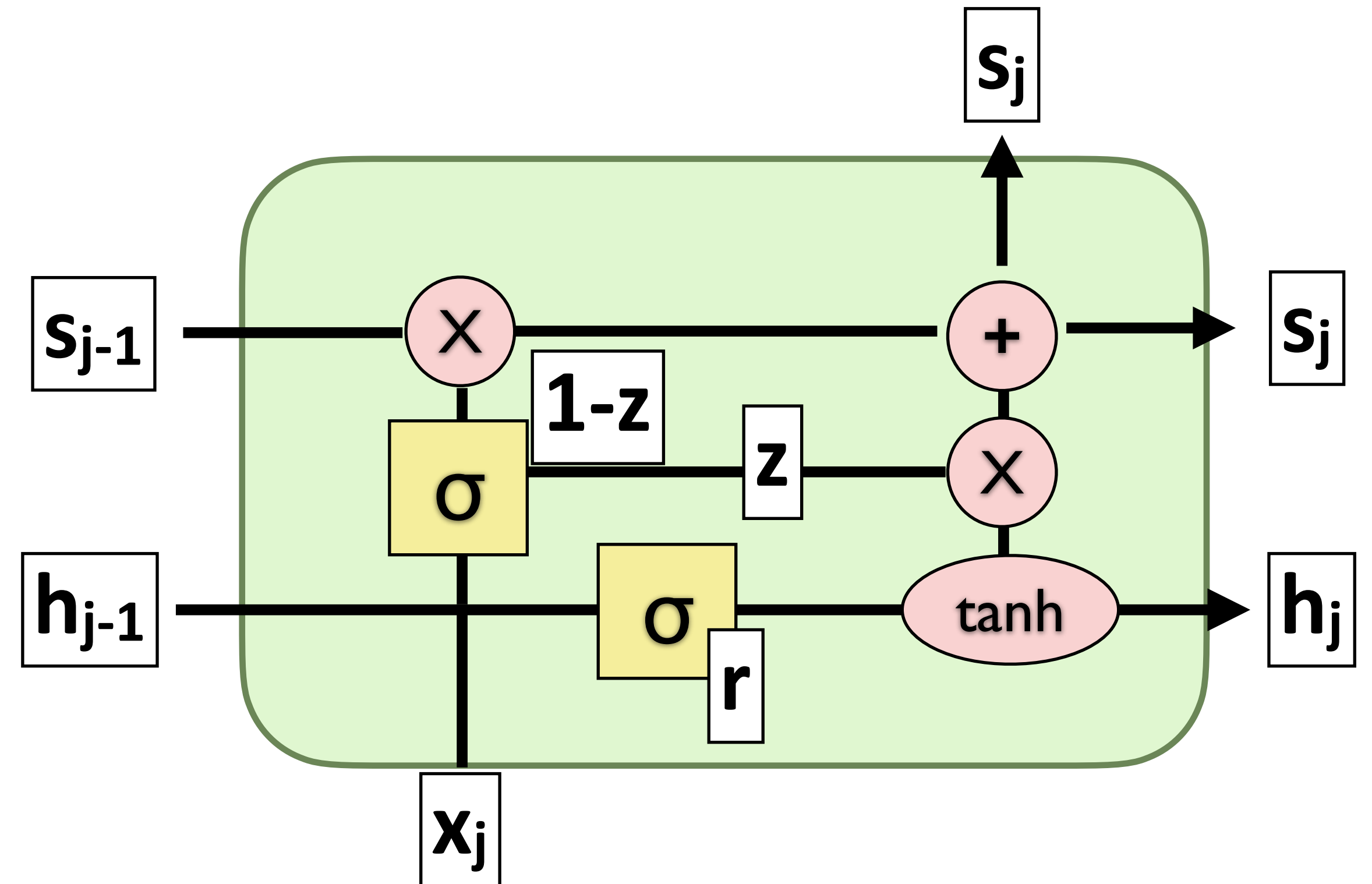
GRU
(Gated Recurrent Unit)



Gated Recurrent Units (GRUs)



- ▶ LSTM: more complex and slower, may work a bit better



- ▶ GRU: faster, a bit simpler
- ▶ Two gates: z (forget, mixes s and h) and r (mixes h and x)

GRUs

- ▶ Also solves the vanishing gradient problem, simpler than LSTM

$$\mathbf{h}_t = (\mathbf{1} - \mathbf{z}) \odot \mathbf{h}_{t-1} + \mathbf{z} \odot \text{func}(\mathbf{x}_t, \mathbf{h}_{t-1})$$

$$\mathbf{z} = \sigma(W\mathbf{x}_t + U\mathbf{h}_{t-1})$$

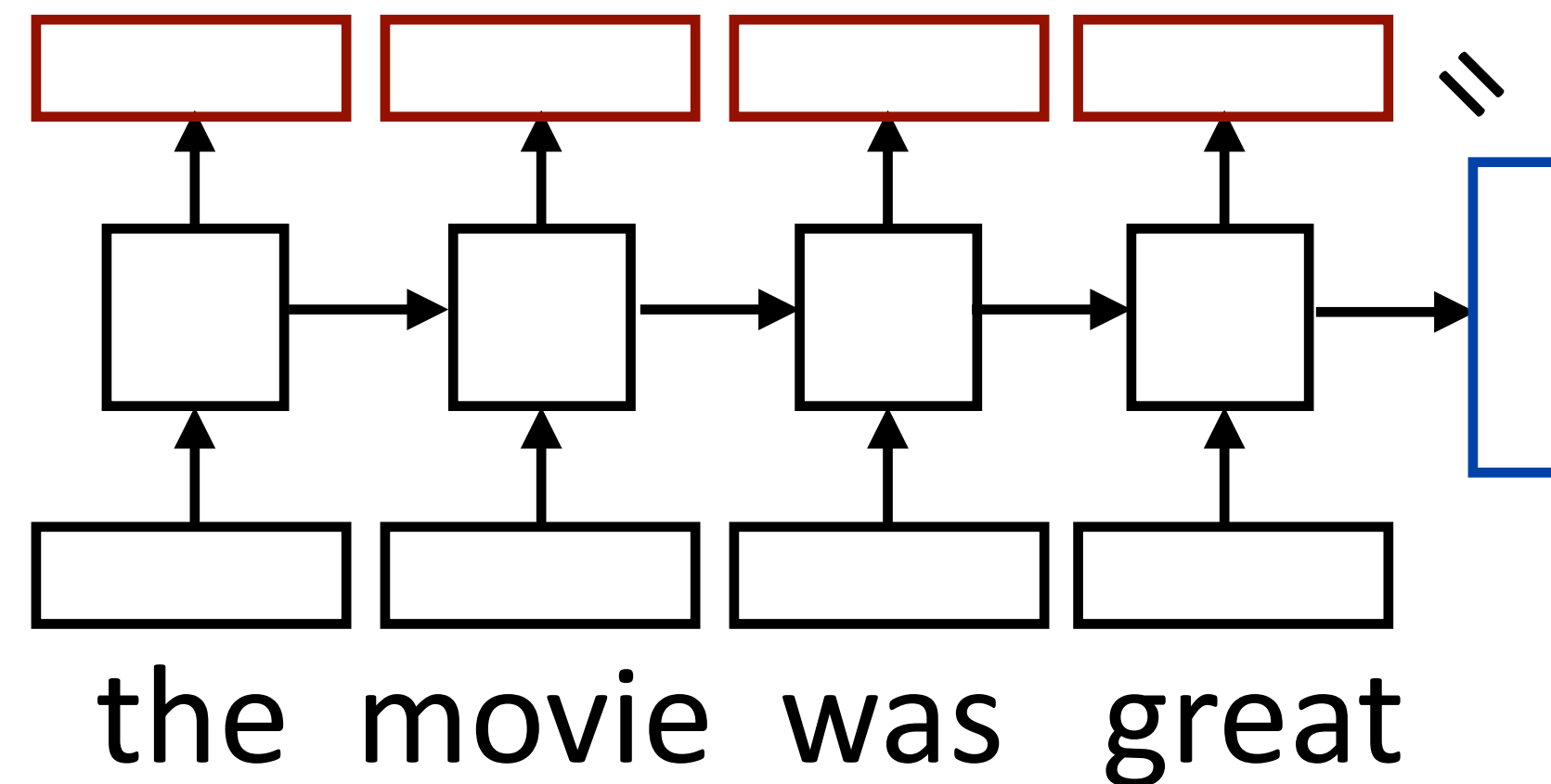
- ▶ $\mathbf{z}$ controls mixing of hidden state $\mathbf{h}$ with new input $\mathbf{x}$
- ▶ Faster to train and sometimes work better (most times not) than LSTMs
- ▶ Other variants of LSTMs:
 - ▶ multiplicative LSTMs (2017), rotational unit of memory (RUM; 2019), ...

Applications

What can LSTMs model?

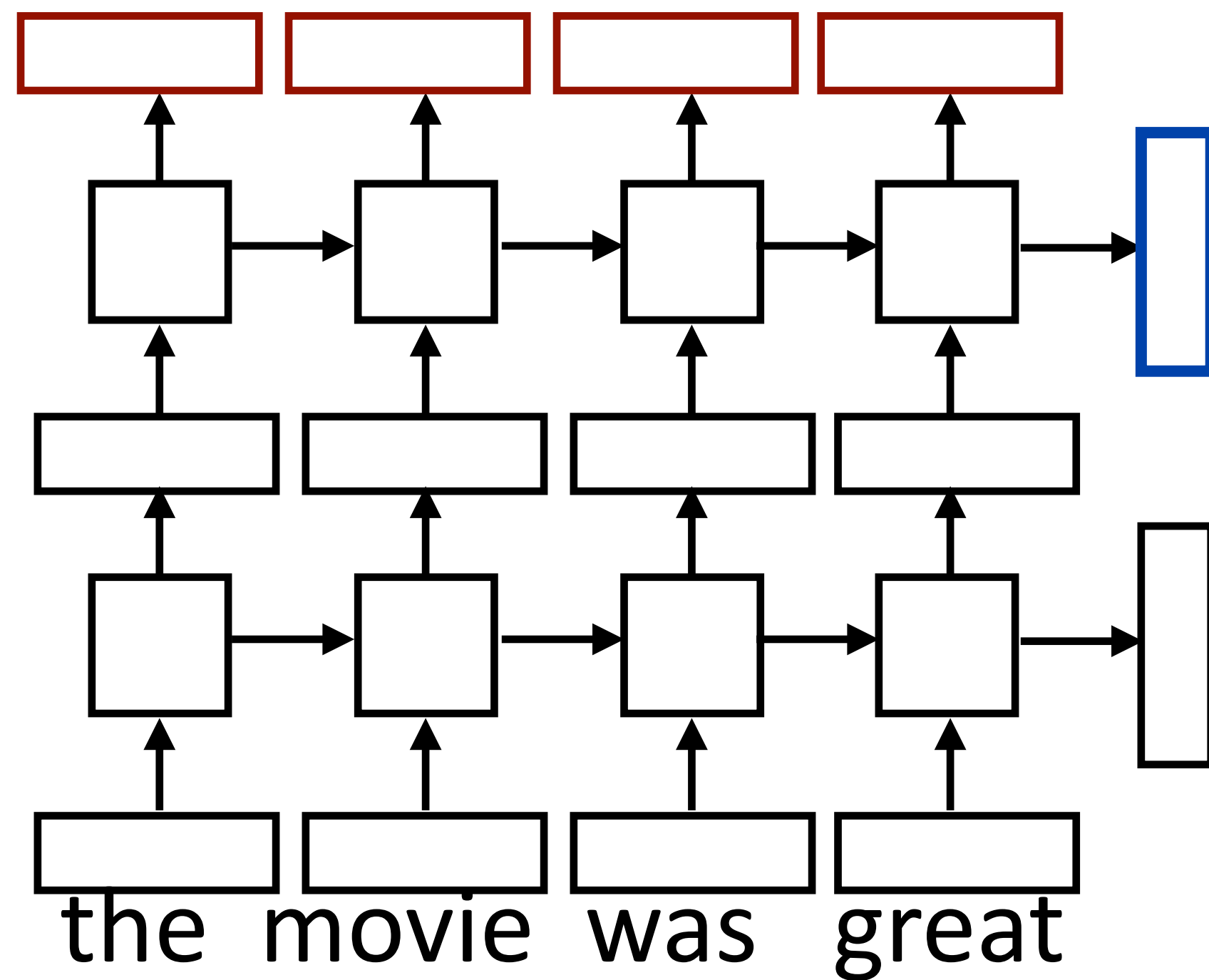
- ▶ Sentence classification (e.g., sentiment)
 - ▶ Encode one sentence, predict
- ▶ Sentence pair classification (e.g., paraphrase identification, NLI)
 - ▶ Encode two sentences, predict
- ▶ Sequential tagging (e.g., POS/NER), or Language models
 - ▶ Move left-to-right, per-token prediction
- ▶ Translation/Generation
 - ▶ Encode sentence + then decode, use token predictions for attention weights (later in the course)

What do RNNs produce?

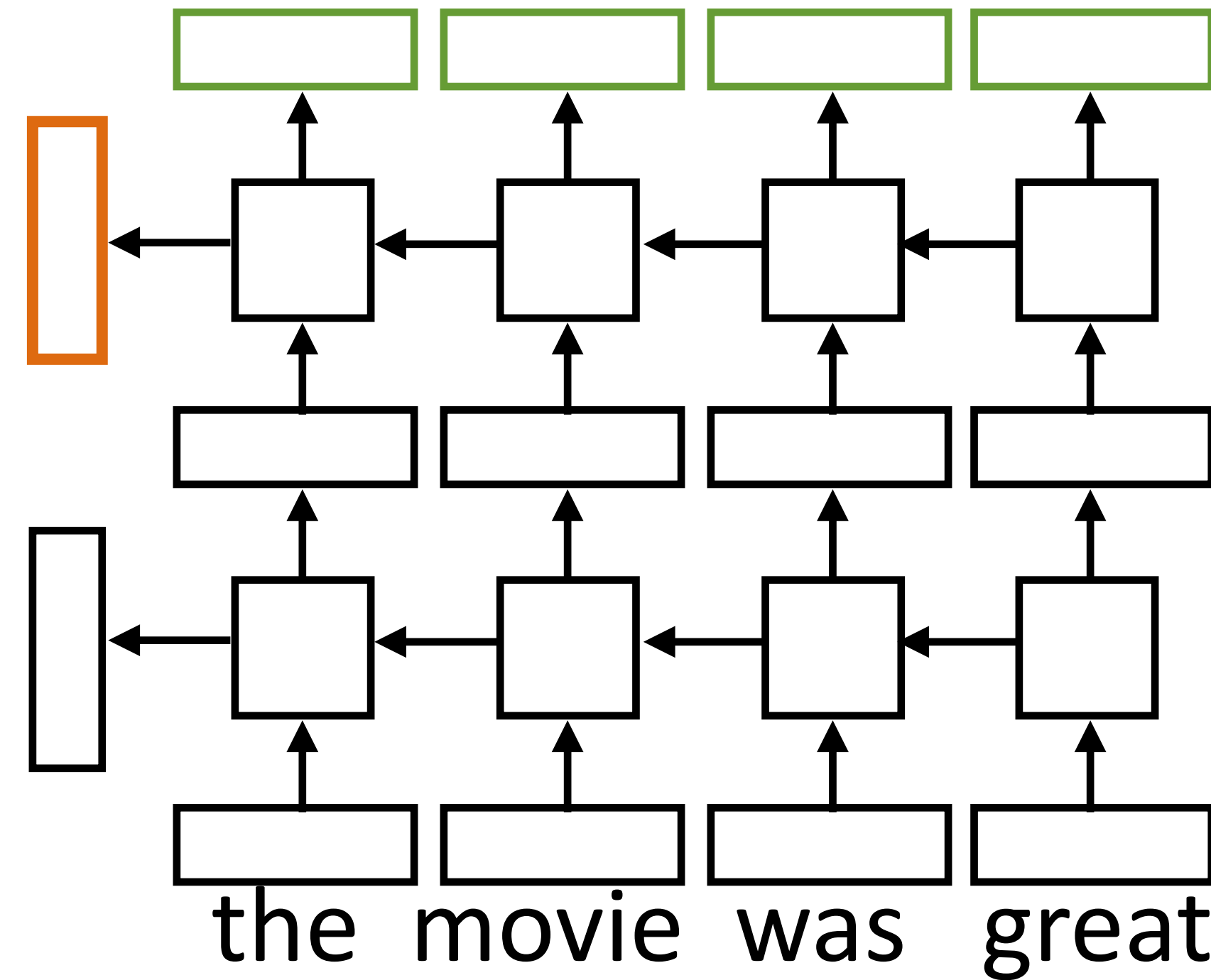


- ▶ **Encoding of the sentence** — can pass this a decoder or make a classification decision about the sentence
- ▶ **Encoding of each word** — can pass this to another layer to make a prediction (can also pool these to get a different sentence encoding)
- ▶ RNN can be viewed as a transformation of a sequence of vectors into a sequence of context-dependent vectors

Multilayer Bidirectional RNN



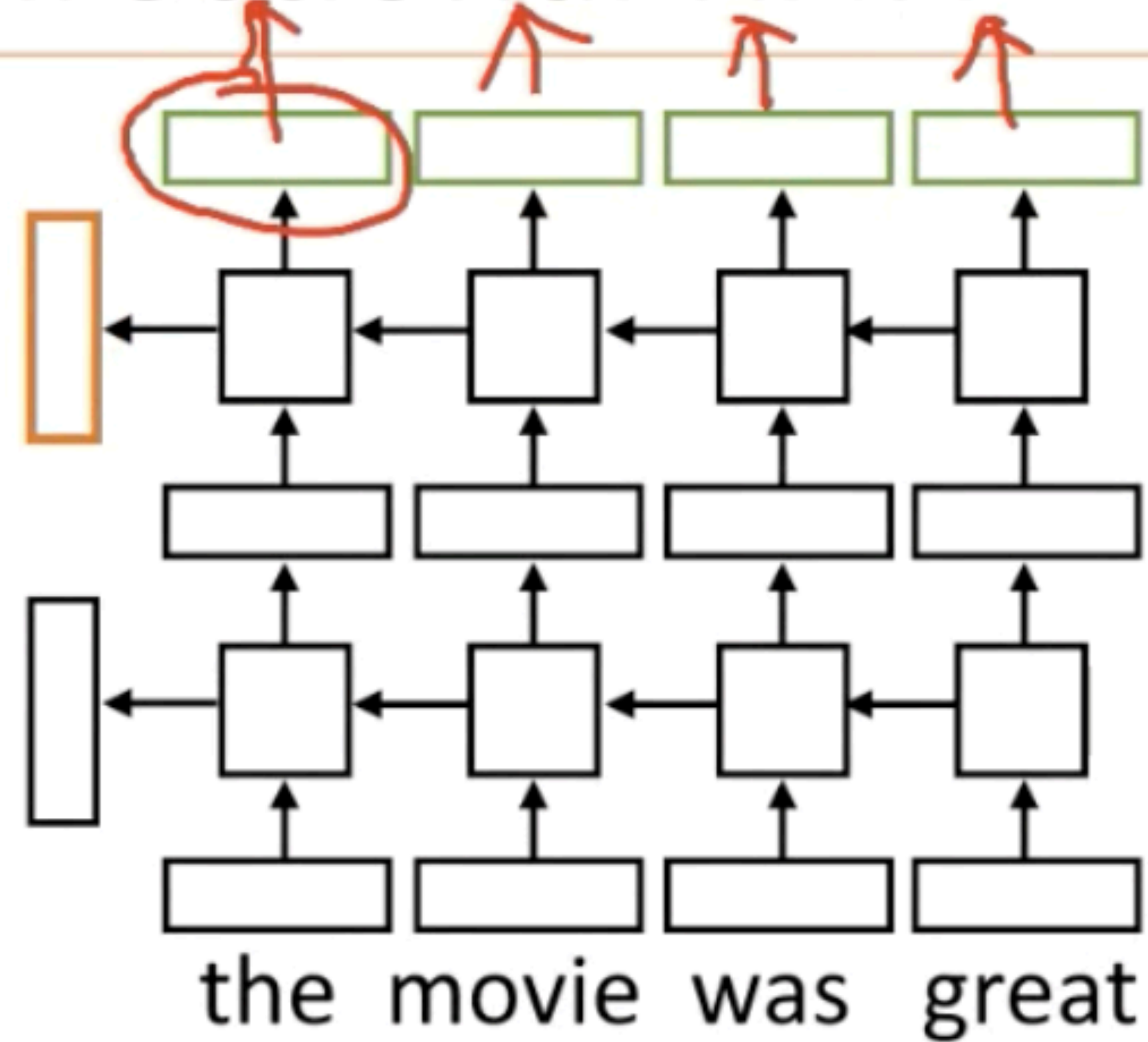
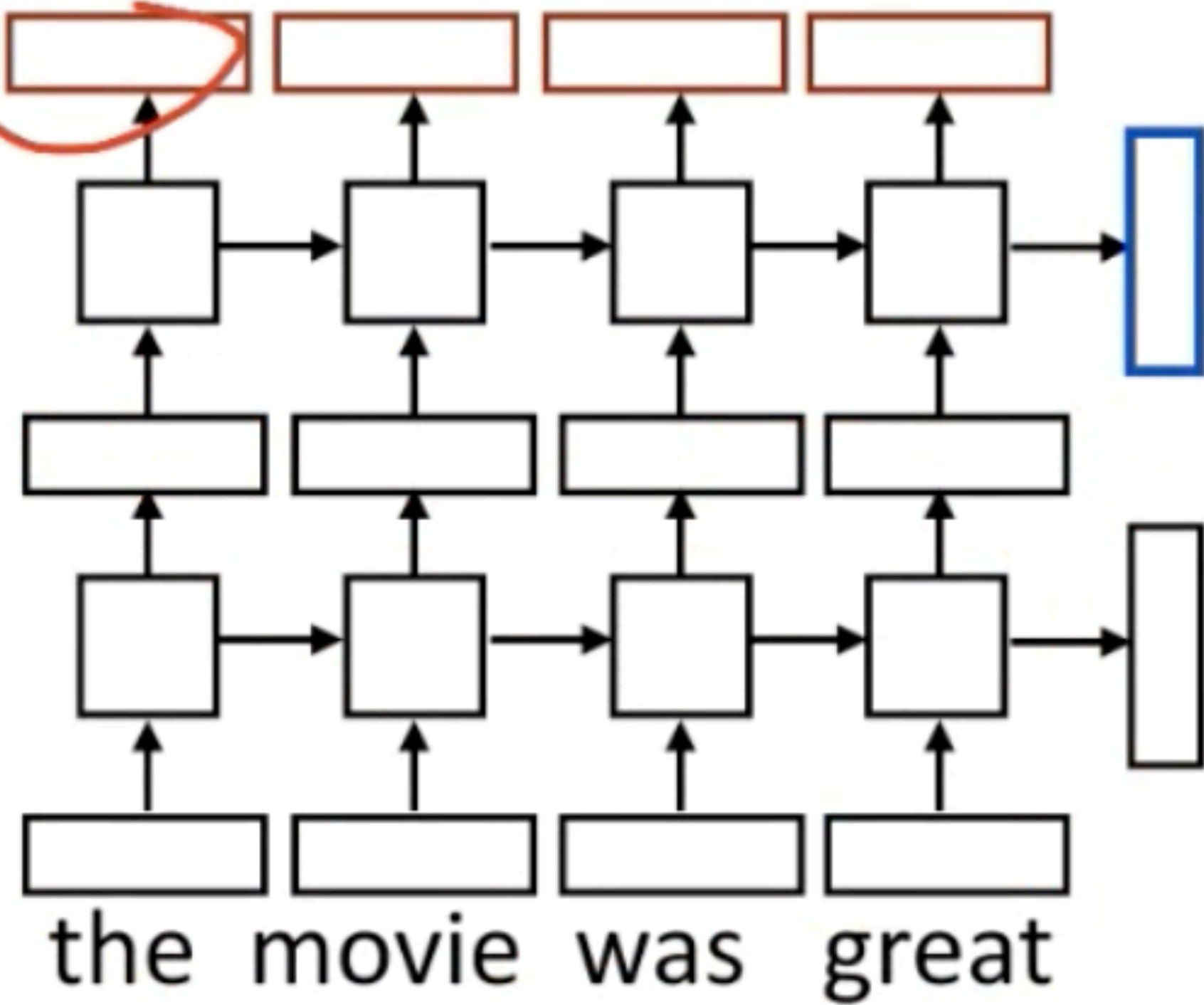
- ▶ Sentence classification based on concatenation of both final outputs



- ▶ Token classification based on concatenation of both directions' token representations



Multilayer Bidirectional RNN



- Sentence classification based on concatenation of both final outputs

$$\text{softmax}(W(h_T \parallel h_1) + b)$$

- Token classification based on concatenation of both directions' token representations

Natural Language Inference

Premise

Hypothesis

A boy plays in the snow

entails

A boy is outside

A man inspects the uniform of a figure

contradicts

The man is sleeping

An older and younger man smiling

neutral

Two men are smiling and
laughing at cats playing

- ▶ Long history of this task: “Recognizing Textual Entailment” challenge in 2006 (Dagan, Glickman, Magnini)
- ▶ Early datasets: small (hundreds of pairs), very ambitious (lots of world knowledge, temporal reasoning, etc.)

SNLI Dataset

- ▶ Show people captions for (unseen) images and solicit entailed / neural / contradictory statements
- ▶ >500,000 sentence pairs

- ▶ Encode each sentence and process

100D LSTM: 78% accuracy

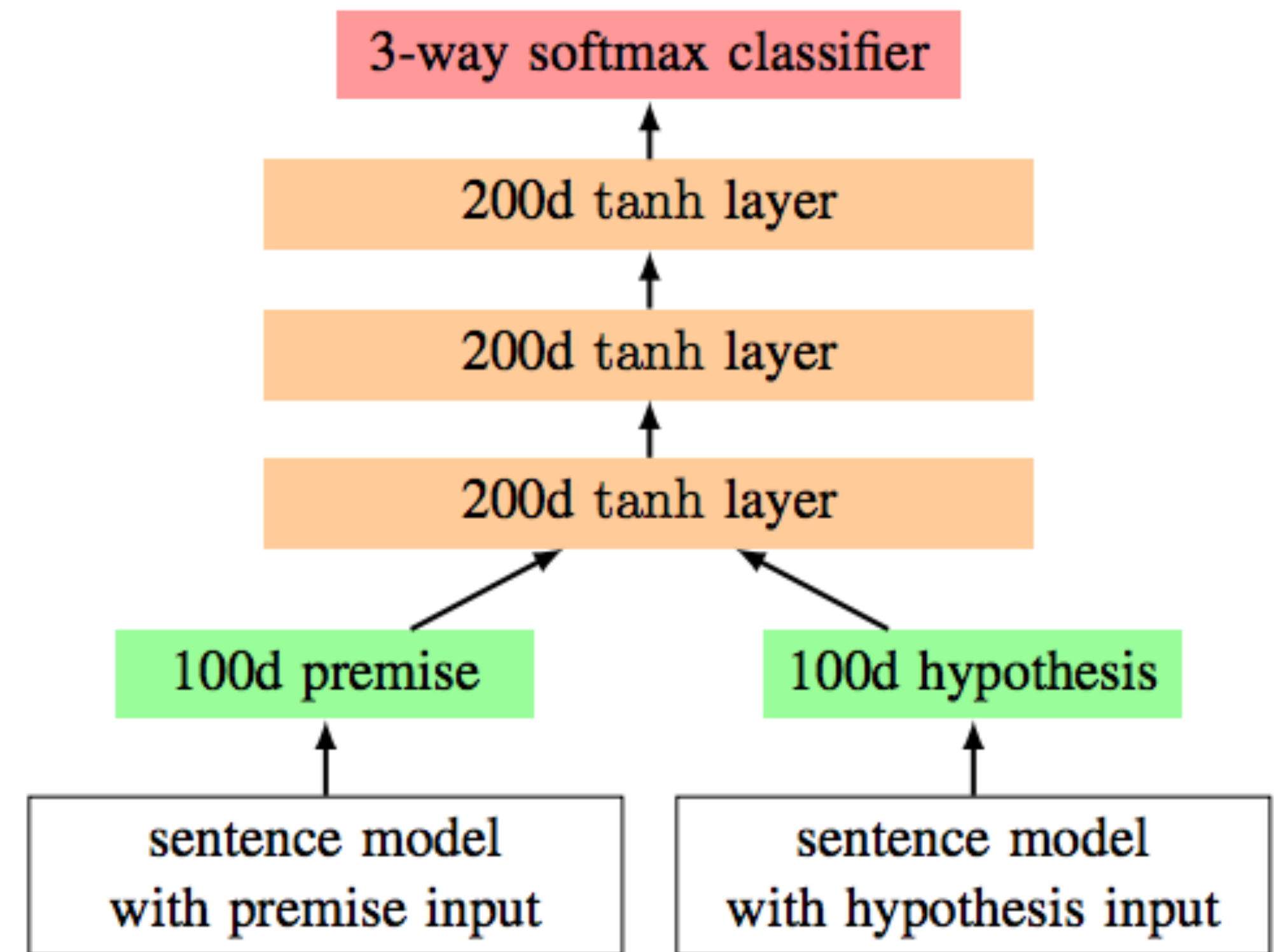
300D LSTM: 80% accuracy

(Bowman et al., 2016)

300D BiLSTM: 83% accuracy

(Liu et al., 2016)

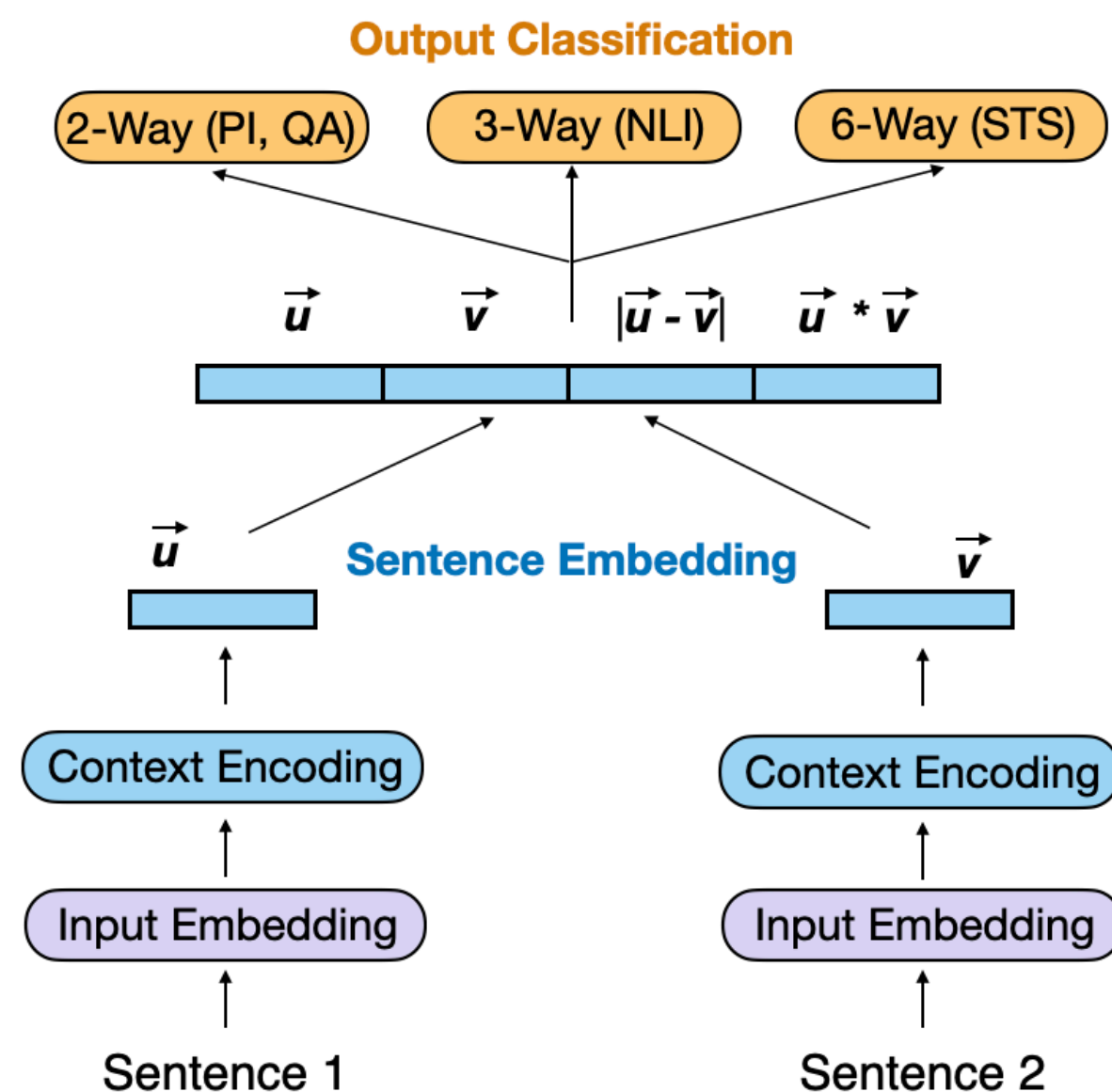
- ▶ Later: better models for this



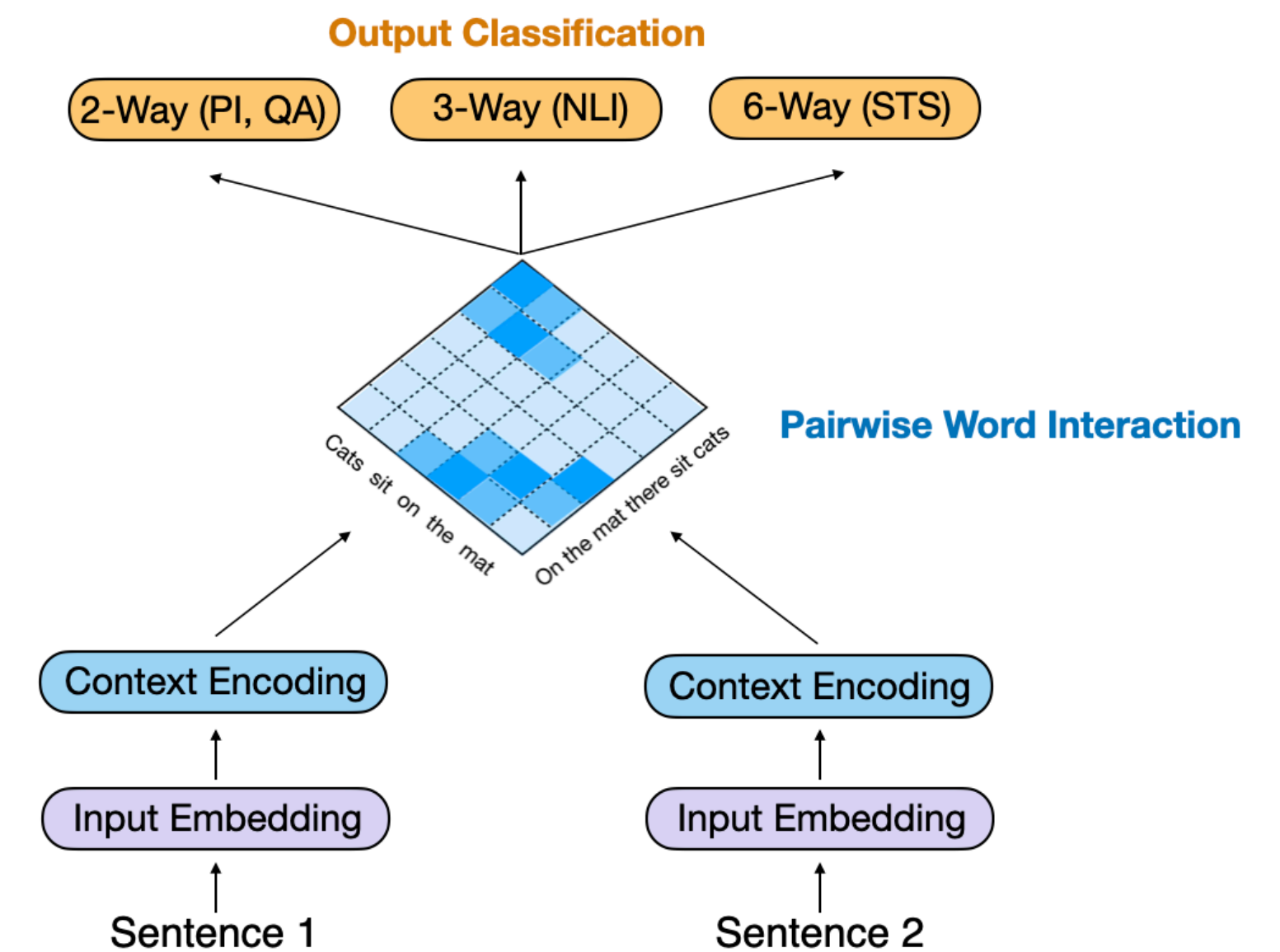
Bowman et al. (2015)

Sentence Pair Classification

Type I: Sentence Encoding-based Models

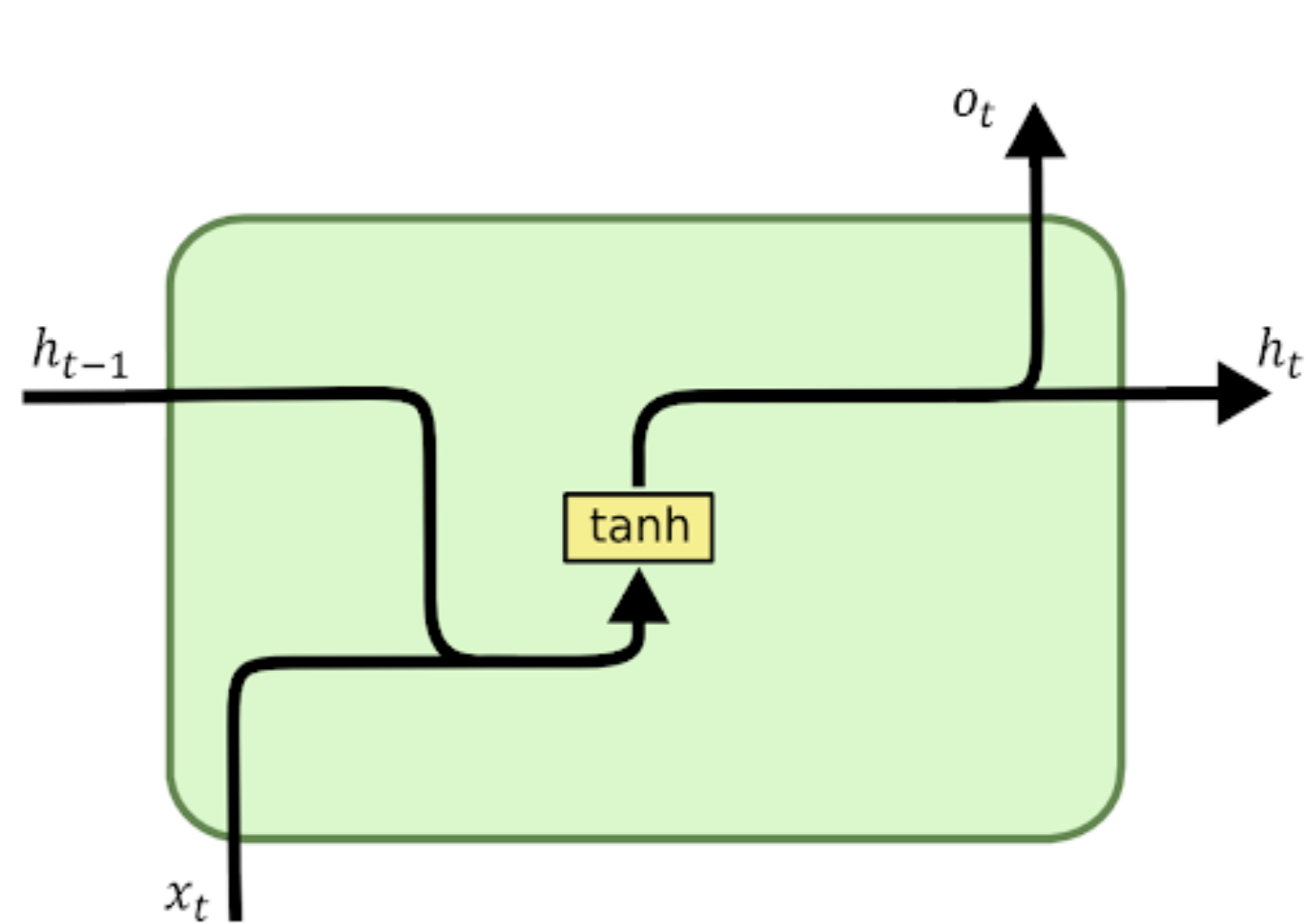


Type II: Word Interaction-based Models

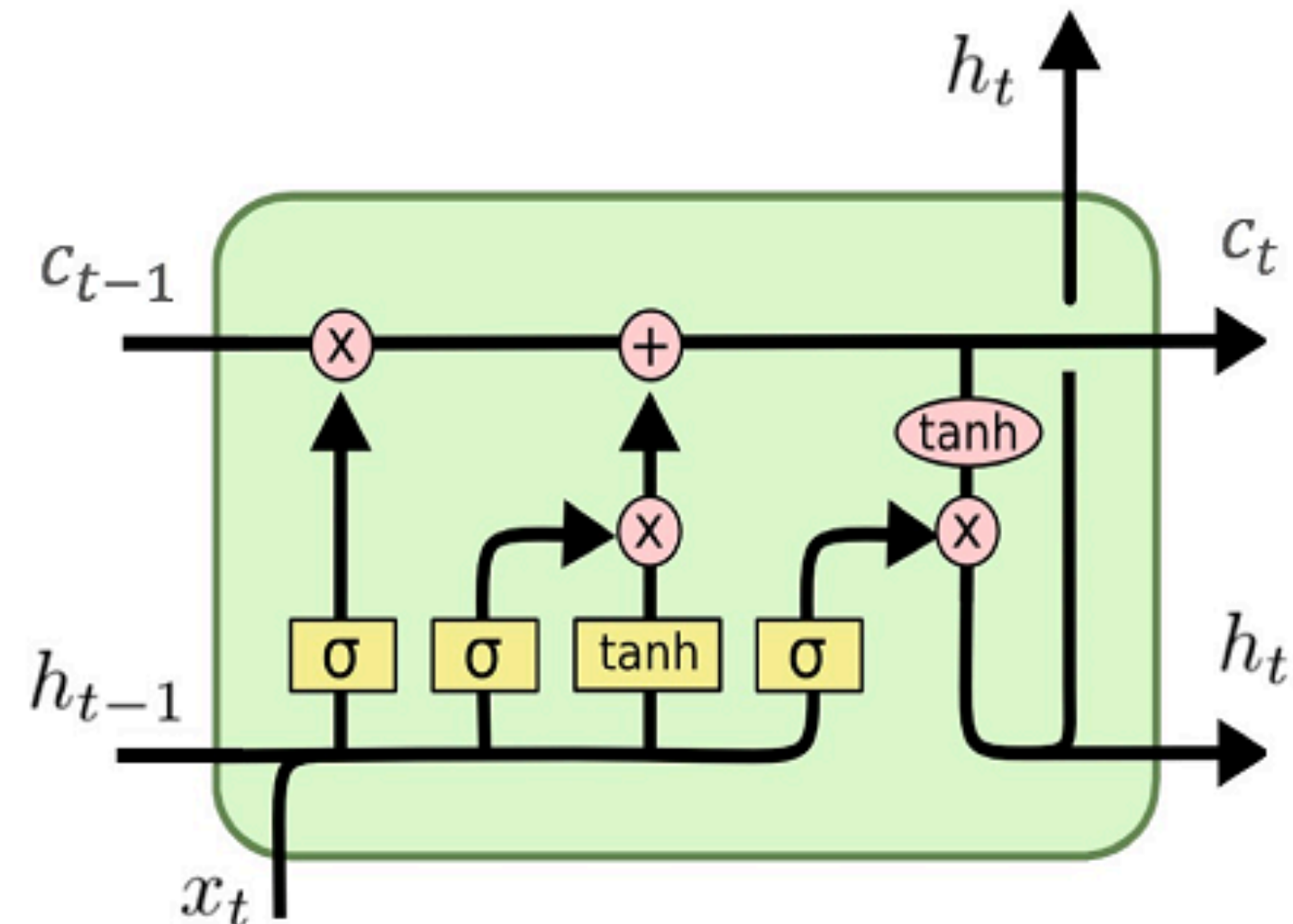


- semantic relation between two sentences depends largely on aligned words/phrases

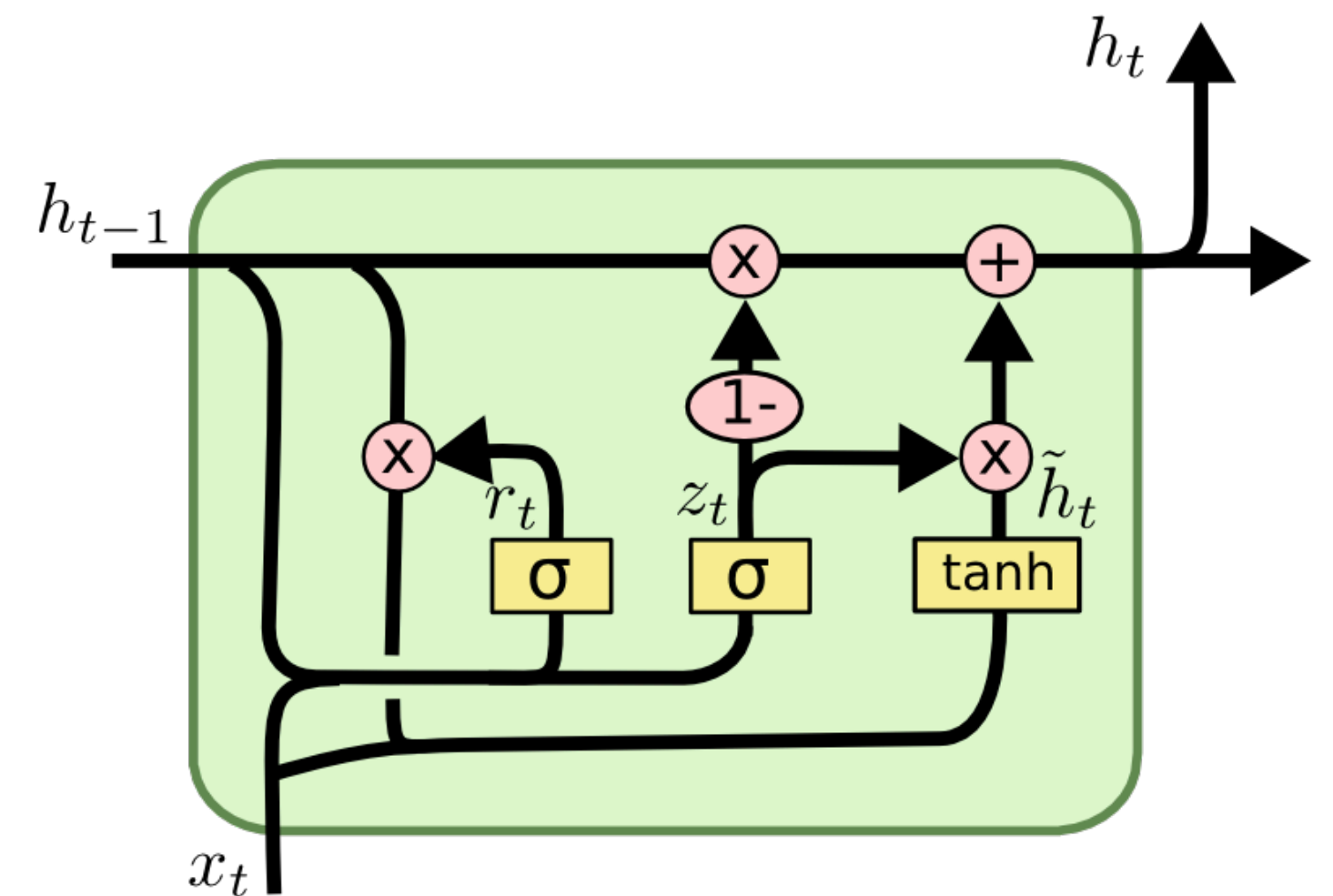
Recap: RNN



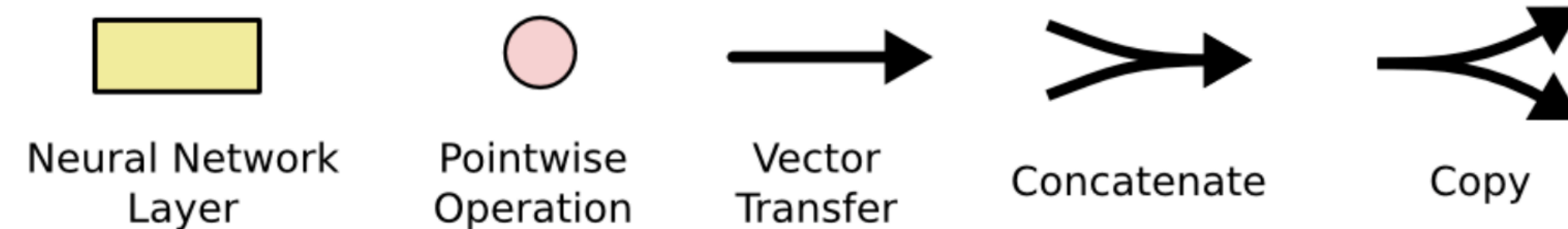
Standard RNN
(with a single layer)



LSTM
(Long Short-term Memory)



GRU
(Gated Recurrent Unit)



Neural CRF

NER Revisited



- ▶ Features in CRFs: $I[\text{tag}=\text{B-LOC} \ \& \ \text{curr_word}=\text{Hangzhou}]$, $I[\text{tag}=\text{B-LOC} \ \& \ \text{prev_word}=\text{to}]$, $I[\text{tag}=\text{B-LOC} \ \& \ \text{curr_prefix}=\text{Han}]$
- ▶ Linear model over features
- ▶ Downsides:
 - ▶ Lexical features mean that words need to be seen in the training data
 - ▶ Linear model can't capture feature conjunctions as effectively (doesn't work well to look at more than 2 words with a single feature)

LSTMs for NER

B-PER I-PER O O O B-LOC O O O B-ORG O O

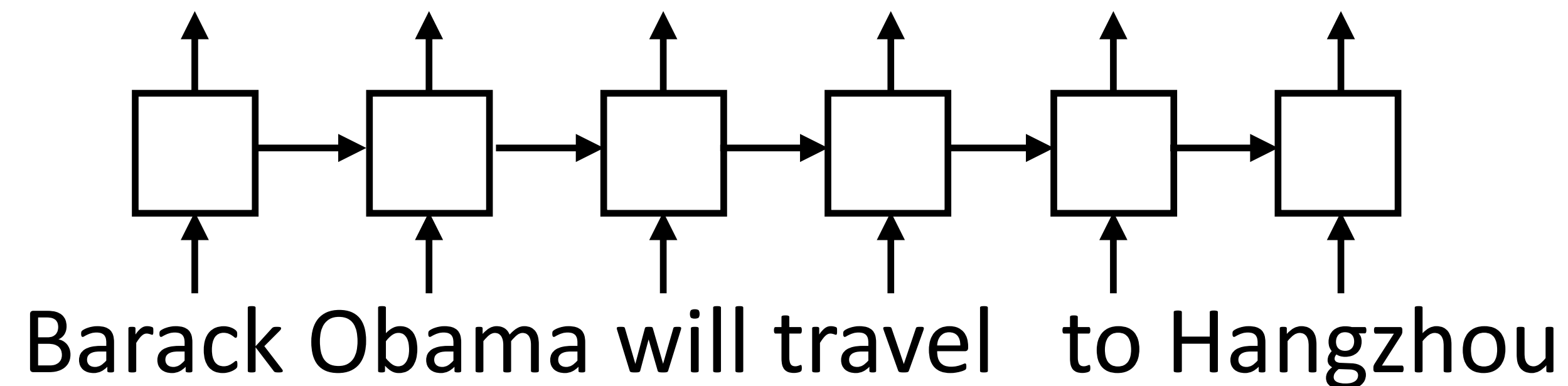
Barack Obama will travel to Hangzhou today for the G20 meeting .

PERSON

LOC

ORG

B-PER I-PER O O O B-LOC



- Transducer (LM-like model)

LSTMs for NER

B-PER I-PER O O O B-LOC O O O B-ORG O O

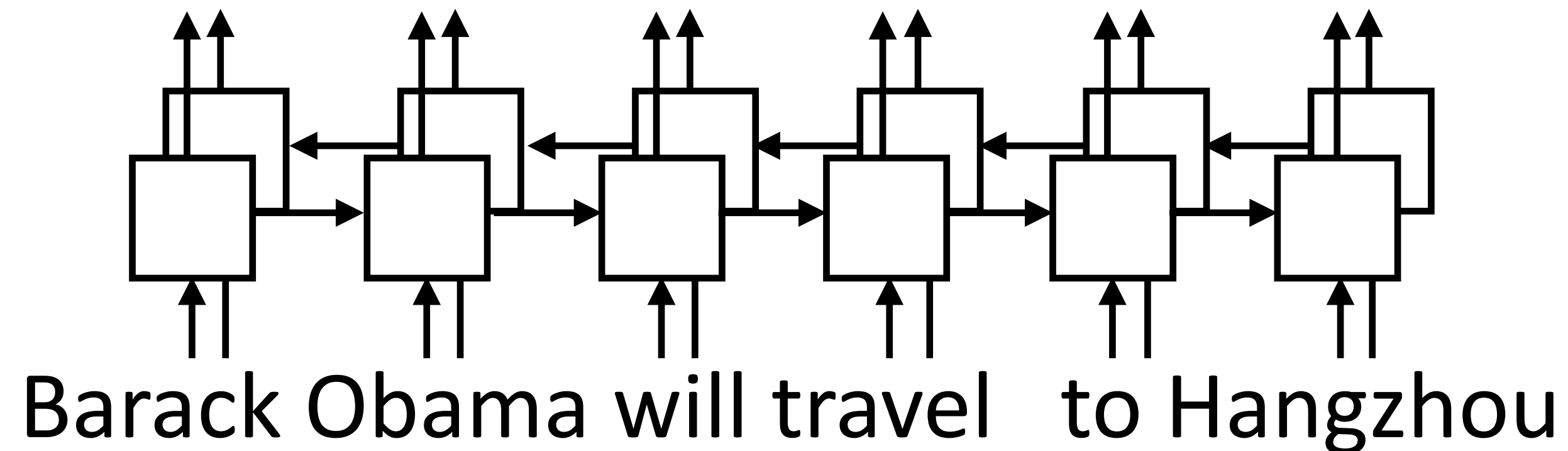
Barack Obama will travel to Hangzhou today for the G20 meeting .

PERSON

LOC

ORG

B-PER I-PER O O O B-LOC



- ▶ Bidirectional transducer model
- ▶ Q: What are the strengths and weaknesses of this model compared to the linear CRFs?

Neural CRFs

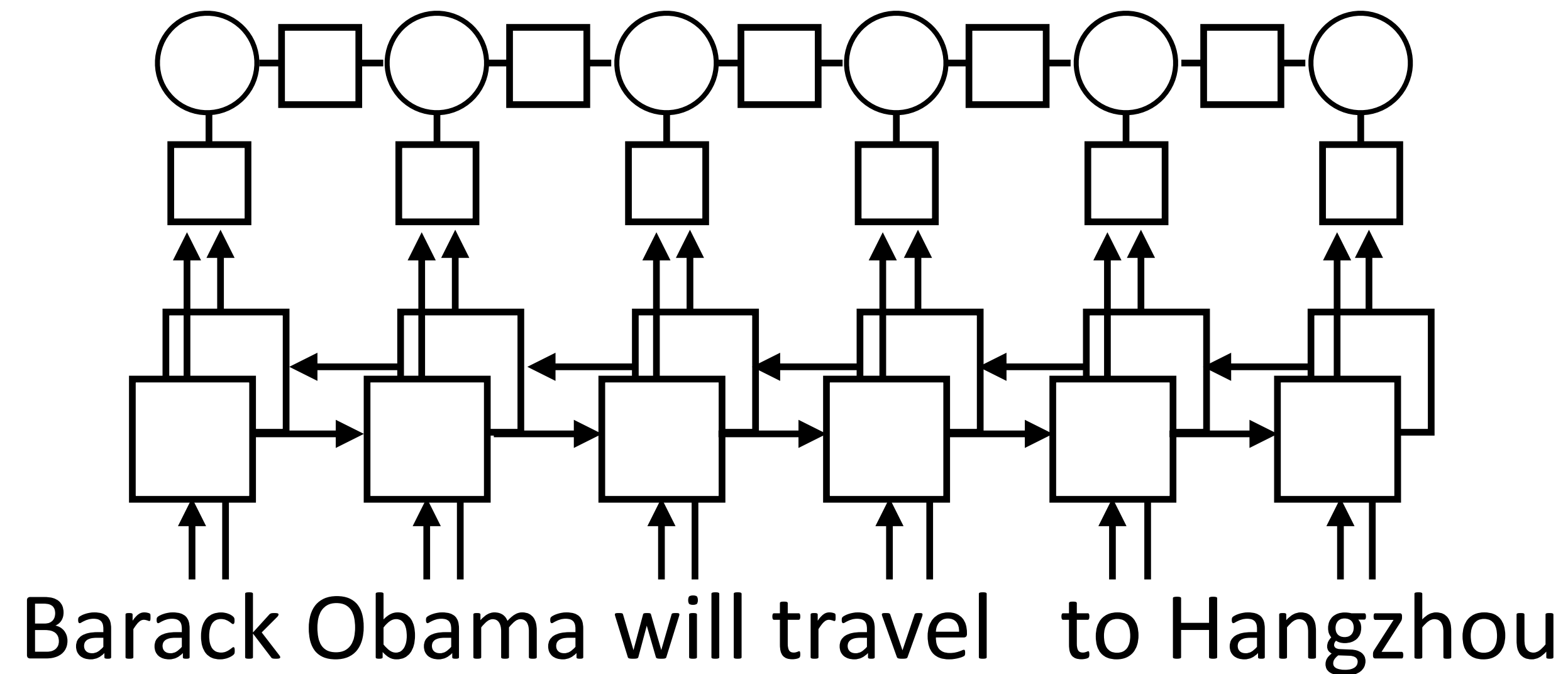
B-PER I-PER O O O B-LOC O O O B-ORG O O

Barack Obama will travel to Hangzhou today for the G20 meeting .

PERSON

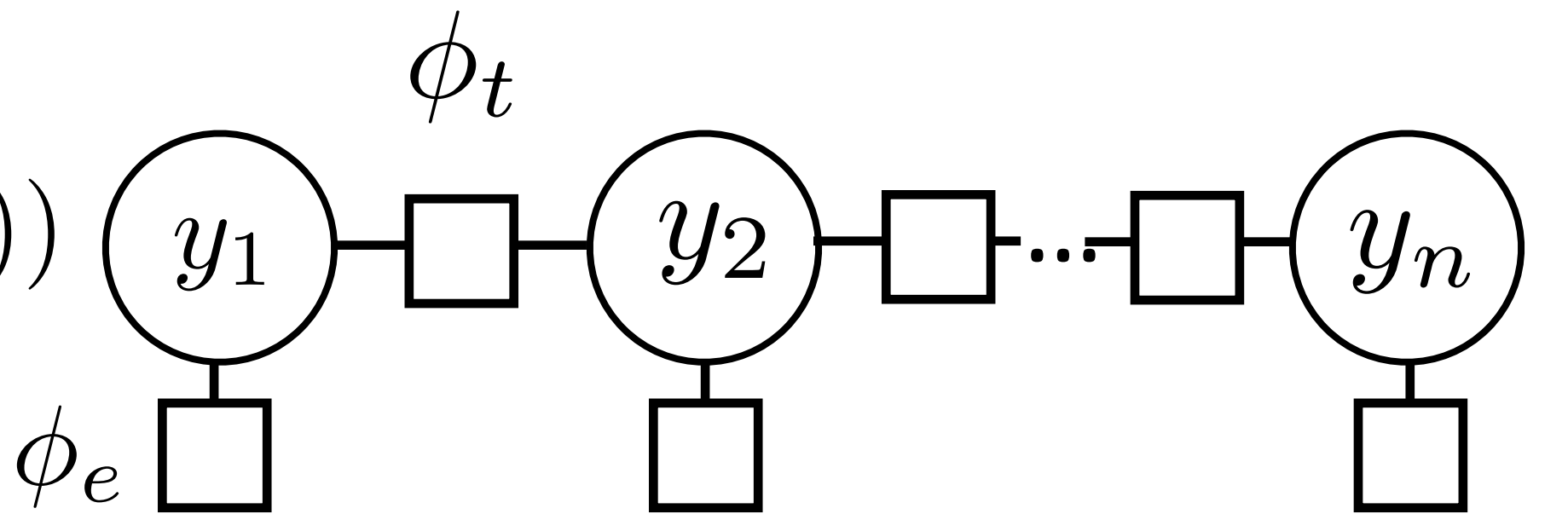
LOC

ORG



- Neural CRFs: bidirectional LSTMs (or some NN) compute emission potentials, capture structural constraints in transition potentials

Neural CRFs

$$P(\mathbf{y}|\mathbf{x}) = \frac{1}{Z} \prod_{i=2}^n \exp(\phi_t(y_{i-1}, y_i)) \prod_{i=1}^n \exp(\phi_e(y_i, i, \mathbf{x}))$$


- ▶ Linear model: $\phi_e(y_i, i, \mathbf{x}) = w^\top f_e(y_i, i, \mathbf{x})$
- ▶ Neural: $\phi_e(y_i, i, \mathbf{x}) = W_{y_i}^\top f(i, \mathbf{x})$ W is a num_tags x len(f) matrix
- ▶ $f(i, \mathbf{x})$ could be the output of a feedforward neural network looking at the words around position i , or the i th output of an LSTM, ...
- ▶ Neural network computes unnormalized potentials that are consumed and “normalized” by a structured model
- ▶ Inference: compute f , use Viterbi

Computing Gradients

$$P(\mathbf{y}|\mathbf{x}) = \frac{1}{Z} \prod_{i=2}^n \exp(\phi_t(y_{i-1}, y_i)) \prod_{i=1}^n \exp(\phi_e(y_i, i, \mathbf{x}))$$

► Conventional: $\phi_e(y_i, i, \mathbf{x}) = w^\top f_e(y_i, i, \mathbf{x})$

► Neural: $\phi_e(y_i, i, \mathbf{x}) = W_{y_i}^\top f(i, \mathbf{x})$

$$\frac{\partial \mathcal{L}}{\partial \phi_{e,i}} = -P(y_i = s|\mathbf{x}) + I[s \text{ is gold}]$$

“error signal”, compute with F-B

► For linear model: $\frac{\partial \phi_{e,i}}{w_i} = f_{e,i}(y_i, i, \mathbf{x})$

chain rule say to multiply together, gives our update

► For neural model: compute gradient of phi w.r.t. parameters of neural net

LSTM Neural CRFs

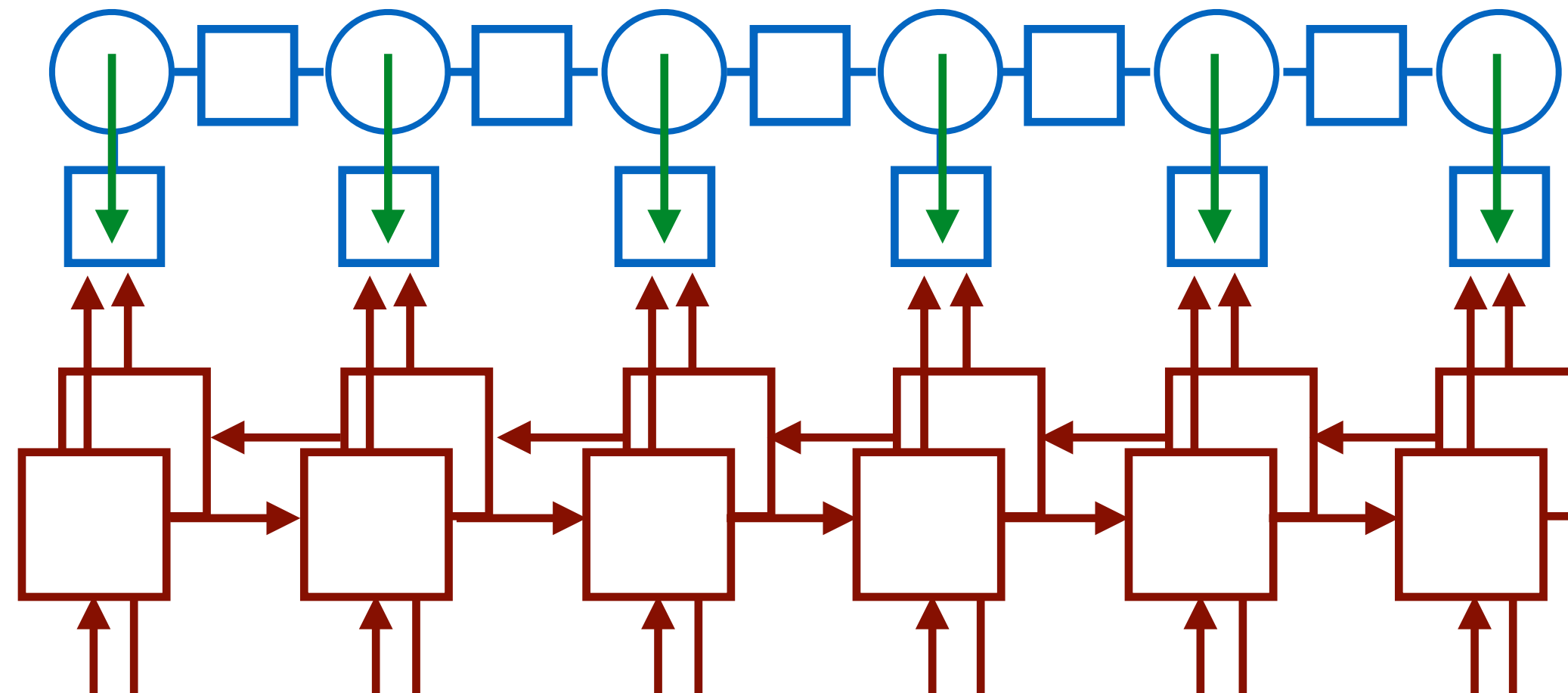
B-PER I-PER O O O B-LOC O O O B-ORG O O

Barack Obama will travel to Hangzhou today for the G20 meeting .

PERSON

LOC

ORG



Barack Obama will travel to Hangzhou

2) Run forward-backward

3) Compute error signal

1) Compute $f(\mathbf{x})$

4) Backprop (no knowledge of sequential structure required)

FFNN Neural CRF for NER

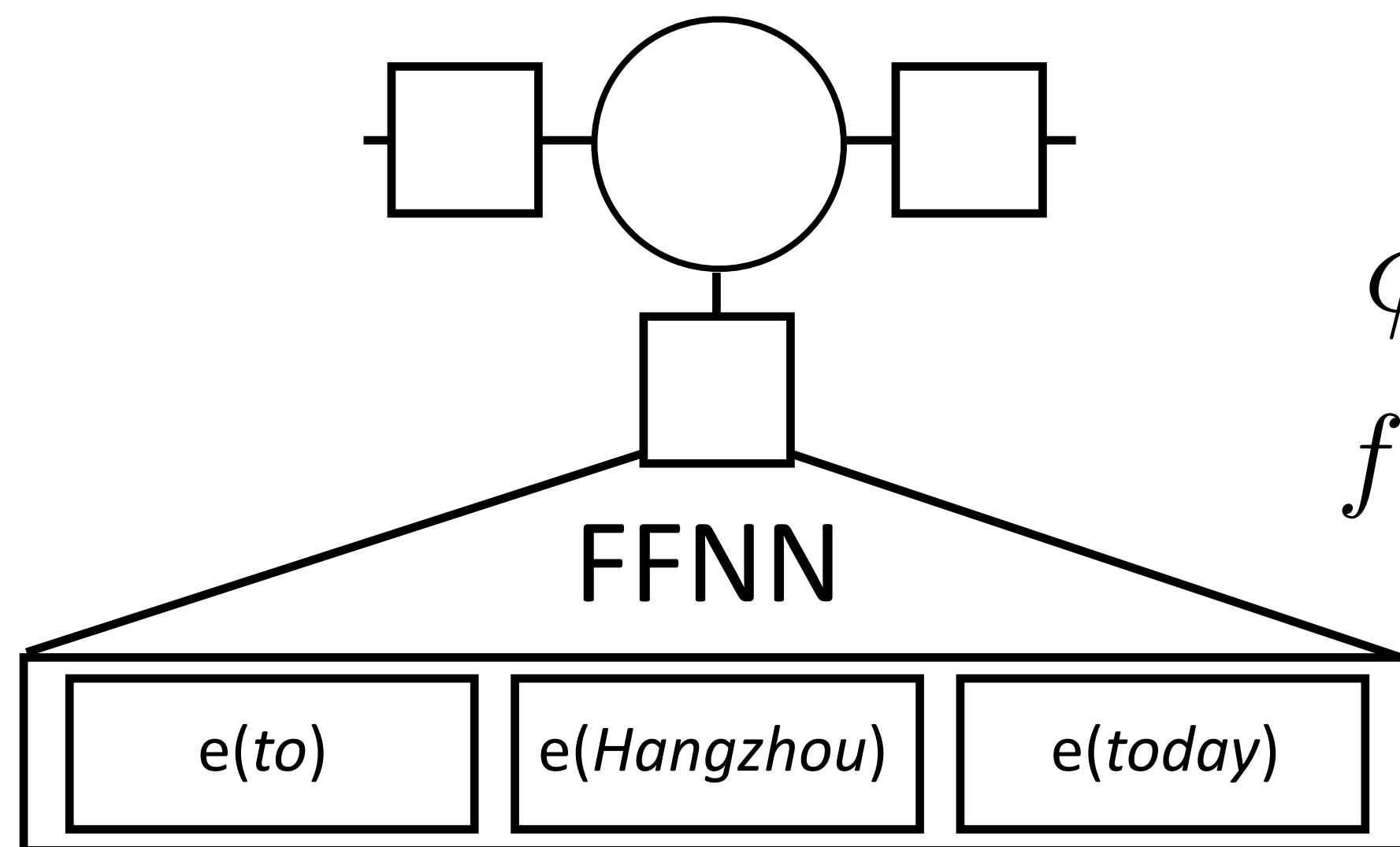
B-PER I-PER O O O B-LOC O O O B-ORG O O

Barack Obama will travel to Hangzhou today for the G20 meeting .

PERSON

LOC

ORG



$$\phi_e = Wg(Vf(\mathbf{x}, i))$$

$$f(\mathbf{x}, i) = [\text{emb}(\mathbf{x}_{i-1}), \text{emb}(\mathbf{x}_i), \text{emb}(\mathbf{x}_{i+1})]$$

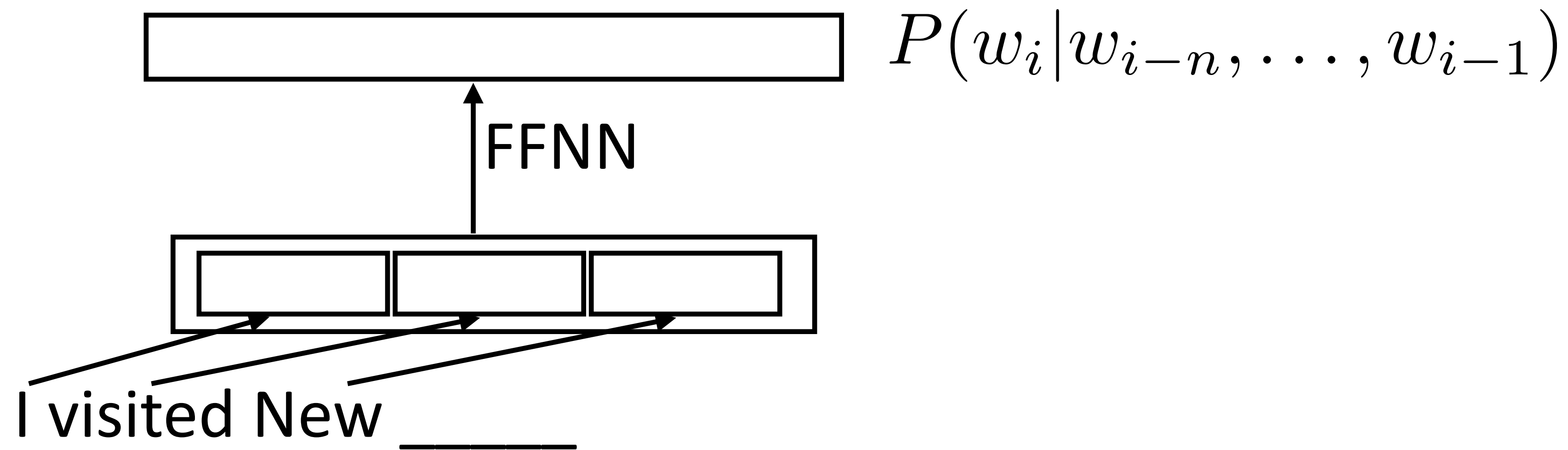
previous word curr word next word

to Hangzhou today

RNN Language Modeling

Neural Language Models

- ▶ Early work: feedforward neural networks looking at context



- ▶ Slow to train over lots of data!
- ▶ Still only look at a fixed window of information...can we use more?

Bengio (2003), Mnih and Hinton (2003)

Neural Probabilistic Language Model

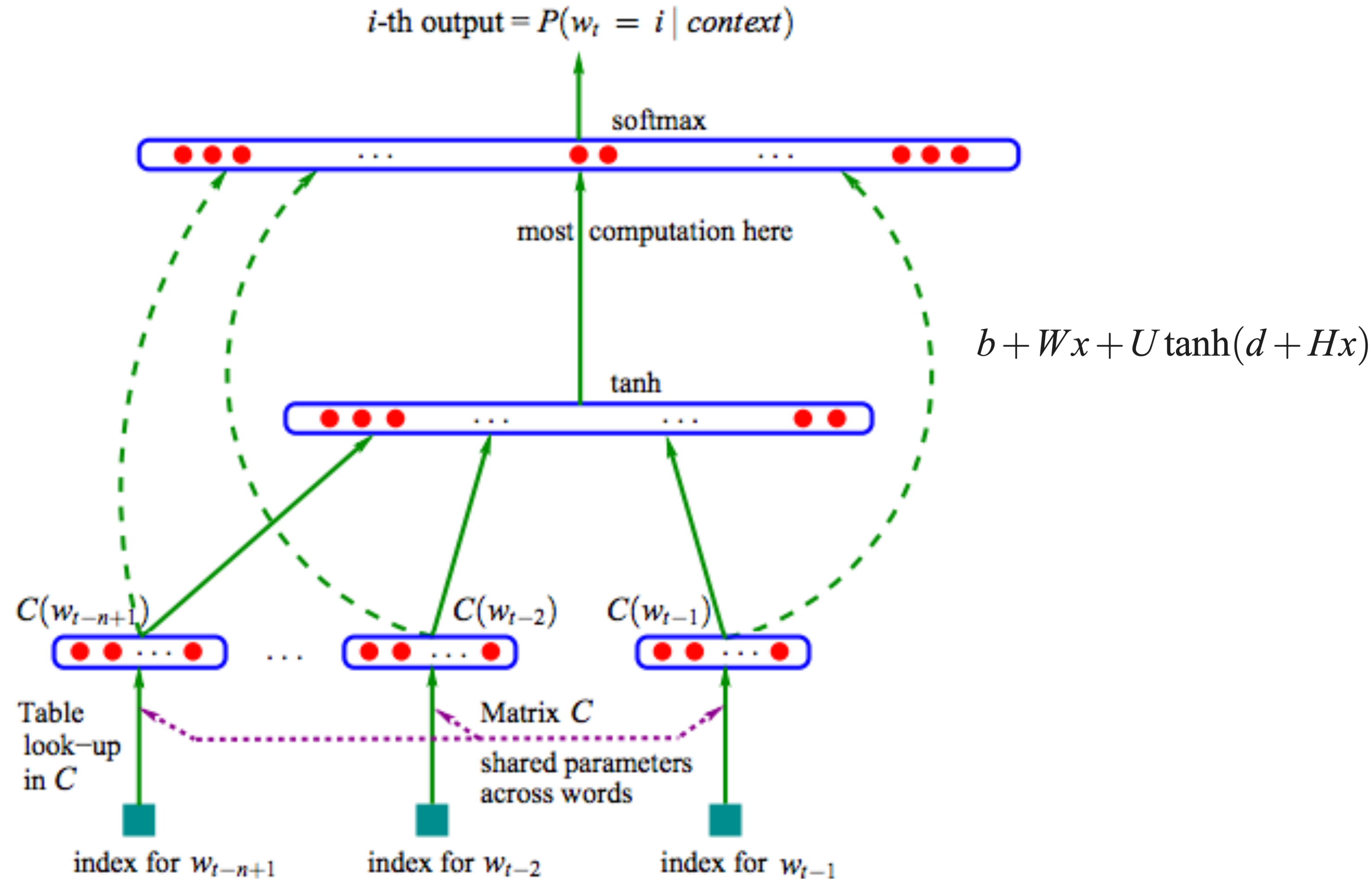
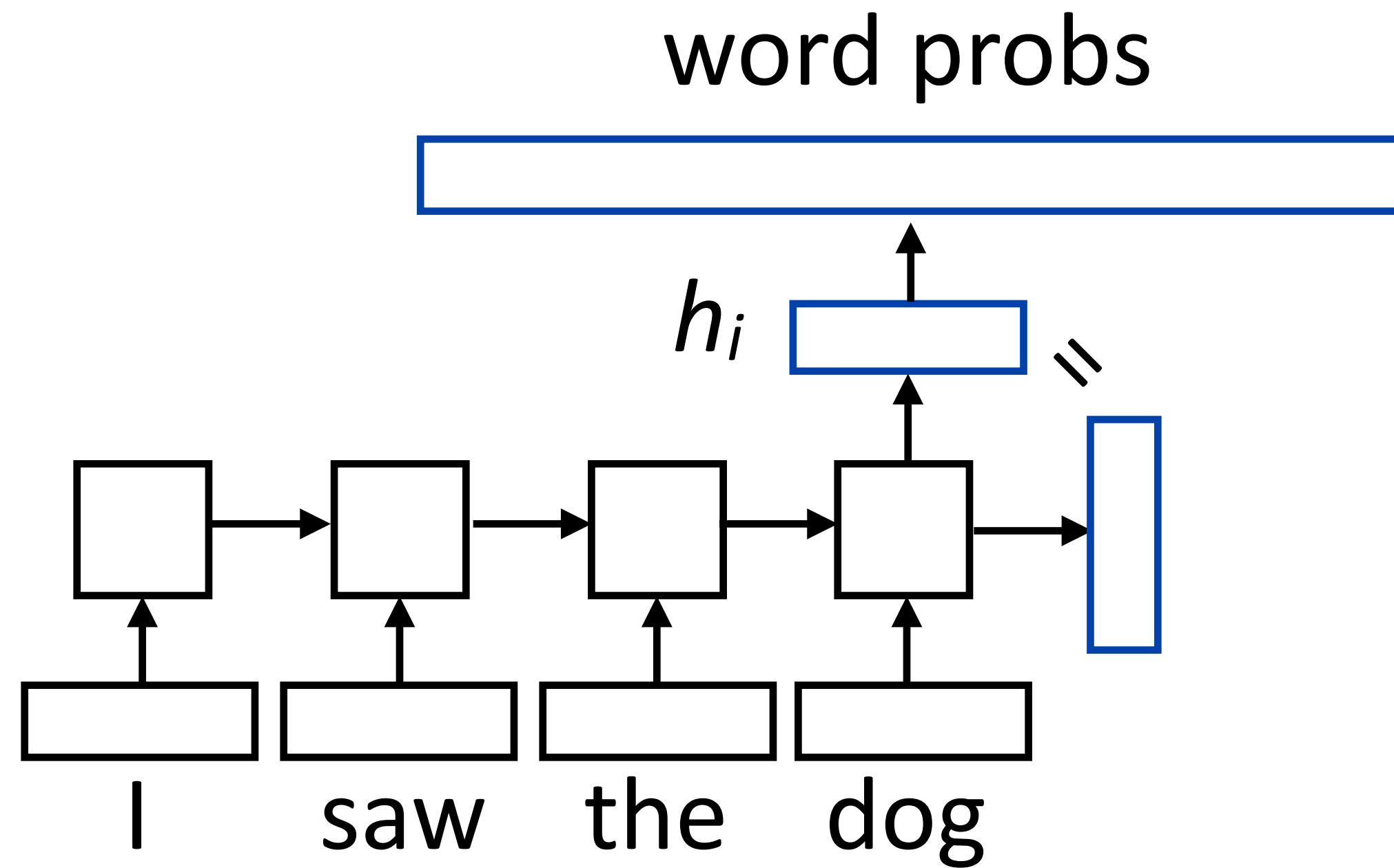


Figure 1: Neural architecture: $f(i, w_{t-1}, \dots, w_{t-n+1}) = g(i, C(w_{t-1}), \dots, C(w_{t-n+1}))$ where g is the neural network and $C(i)$ is the i -th word feature vector.

Bengio et al. (2003)

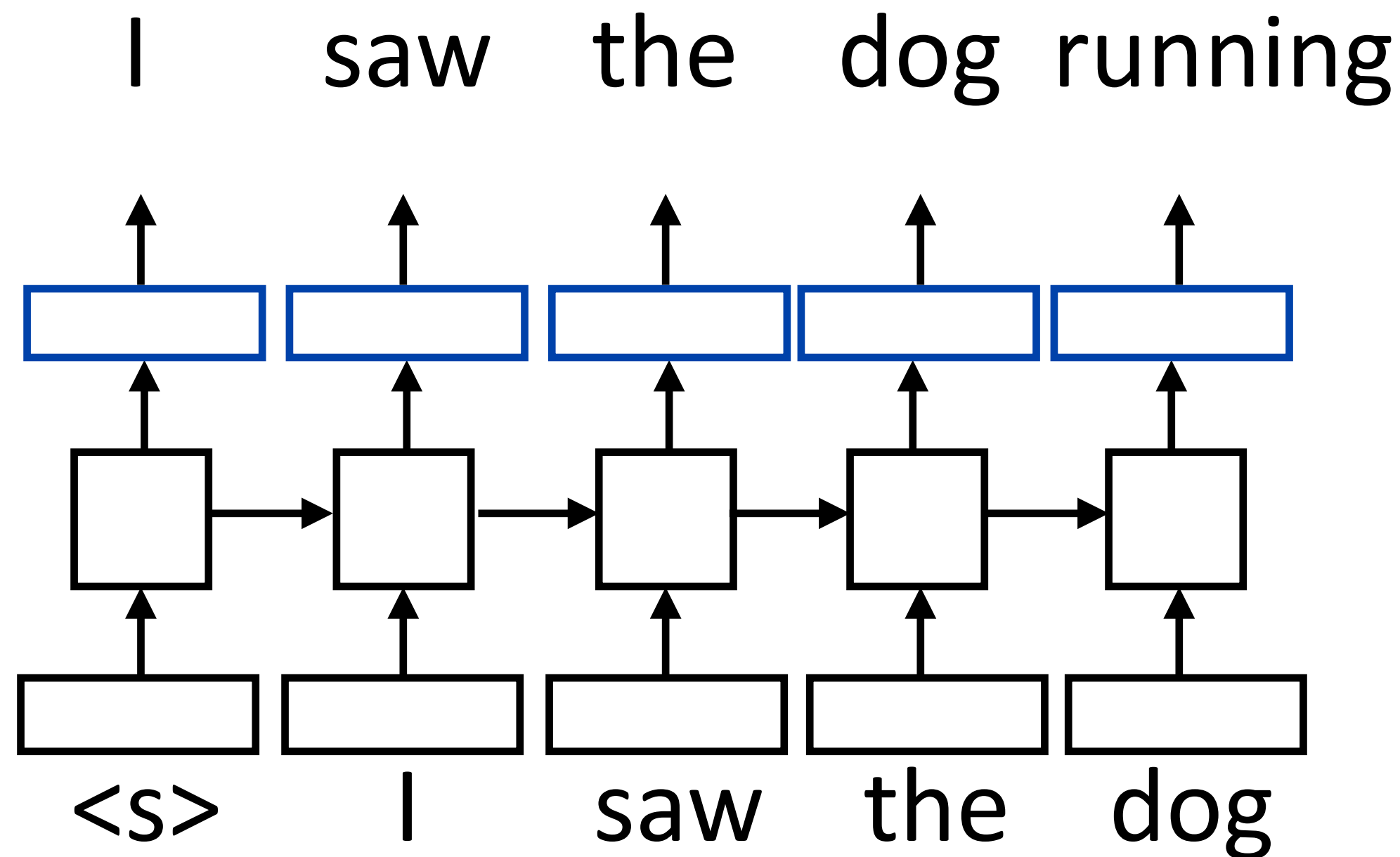
RNN Language Modeling



$$P(w|\text{context}) = \text{softmax}(W\mathbf{h}_i)$$

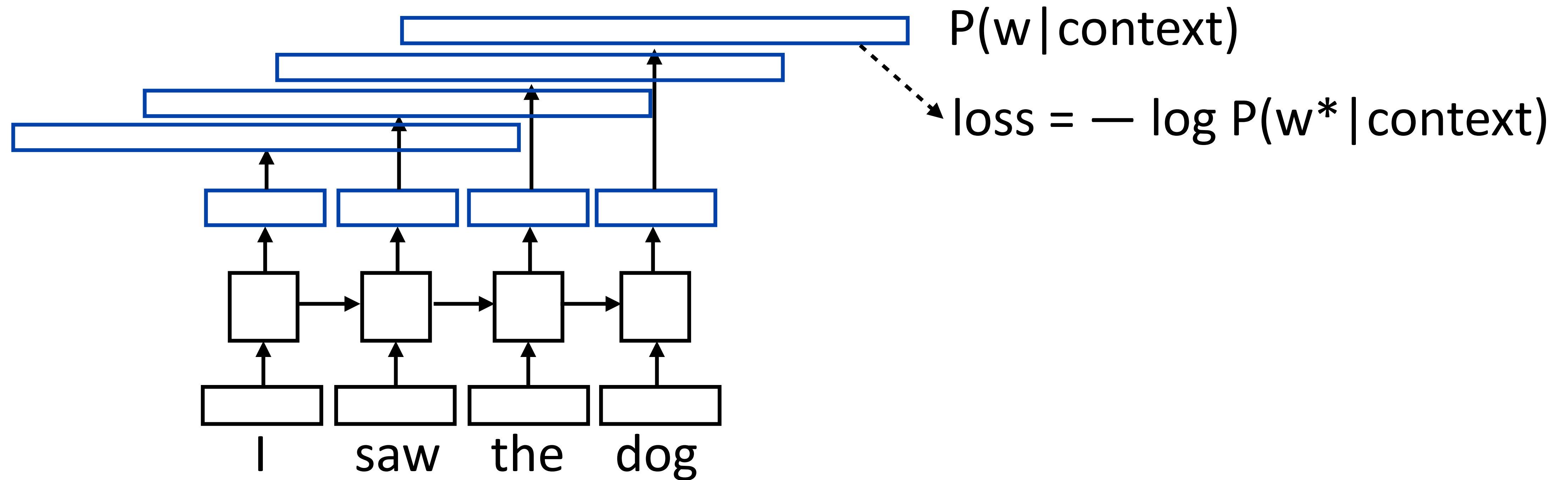
- ▶ W is a (vocab size) x (hidden size) matrix

Training RNNLMs



- ▶ Input is a sequence of words, output is those words shifted by one,
- ▶ Allows us to efficiently batch up training across time (one run of the RNN)

Training RNNLMs

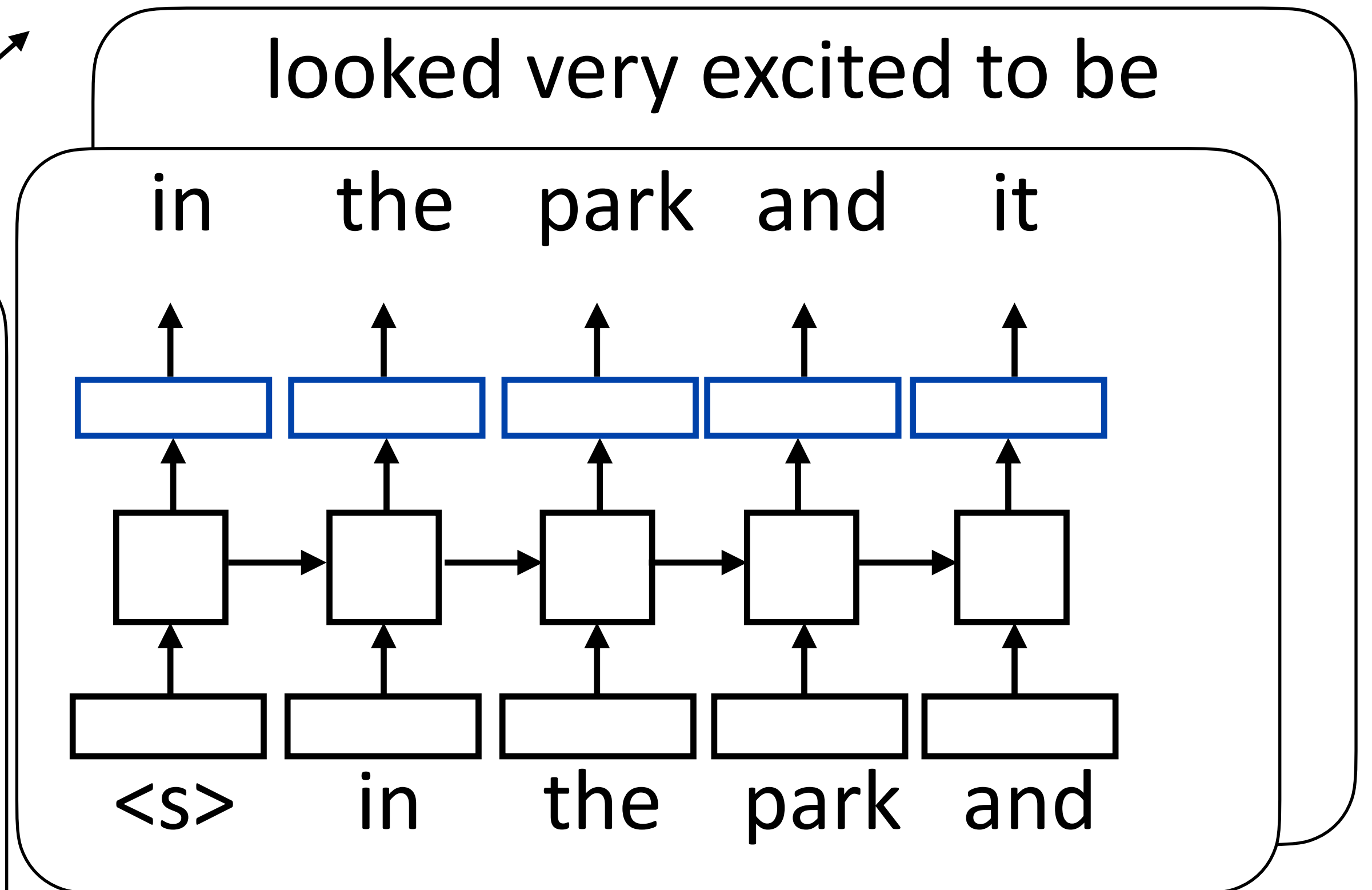
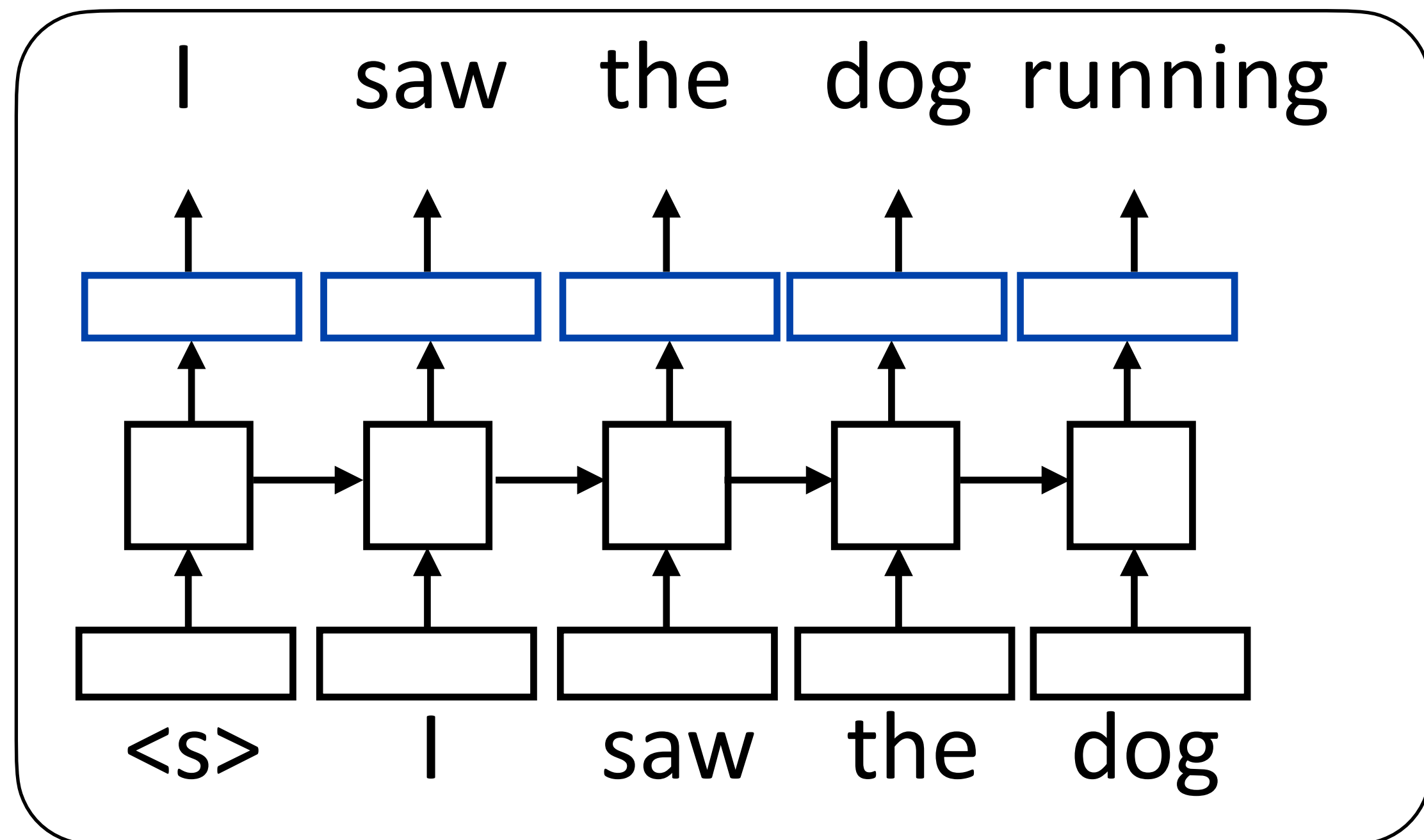


- ▶ Total loss = sum of negative log likelihoods at each position
- ▶ Backpropagate through the network to simultaneously learn to predict next word given previous words at all positions

Batched LM Training

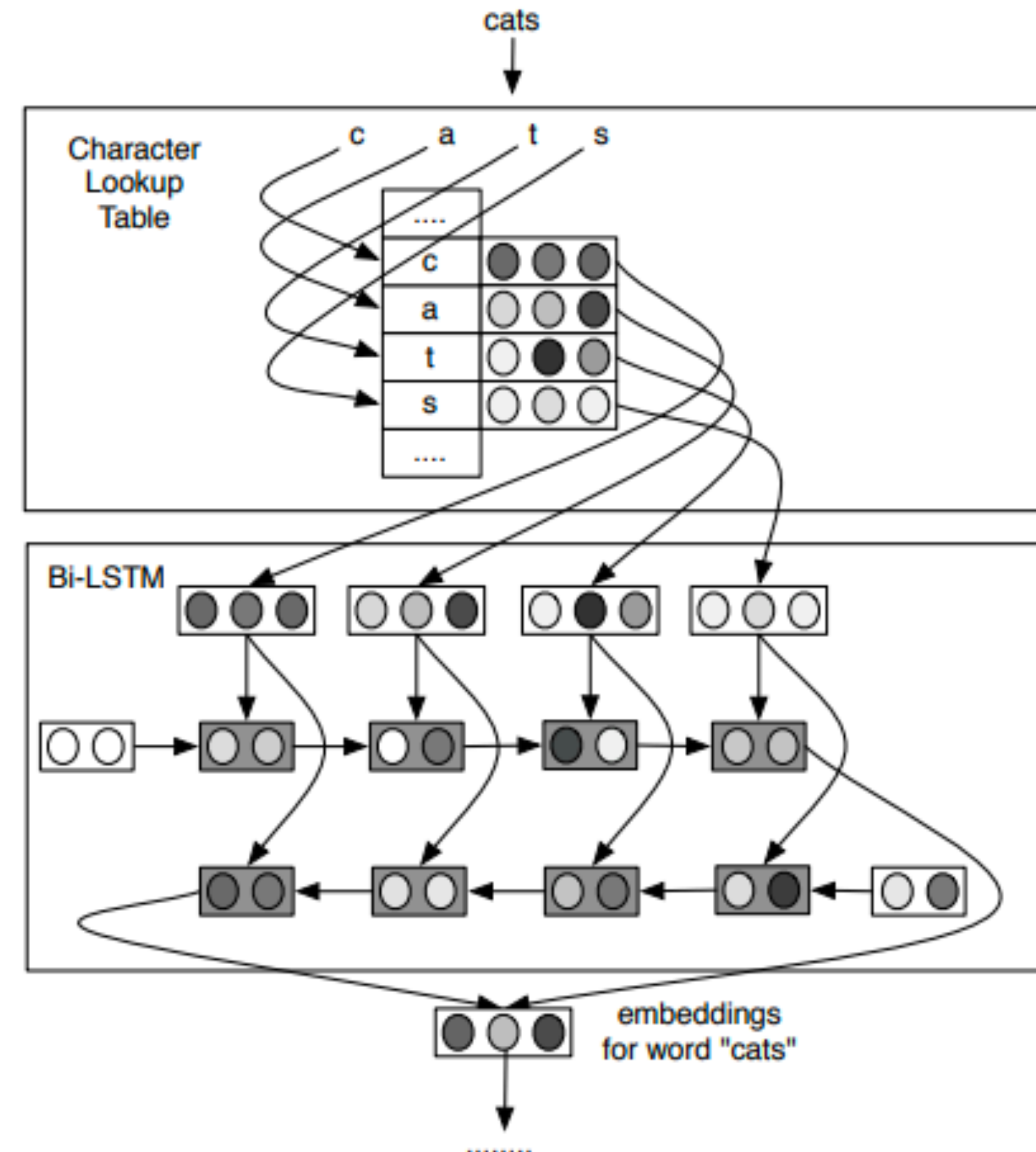
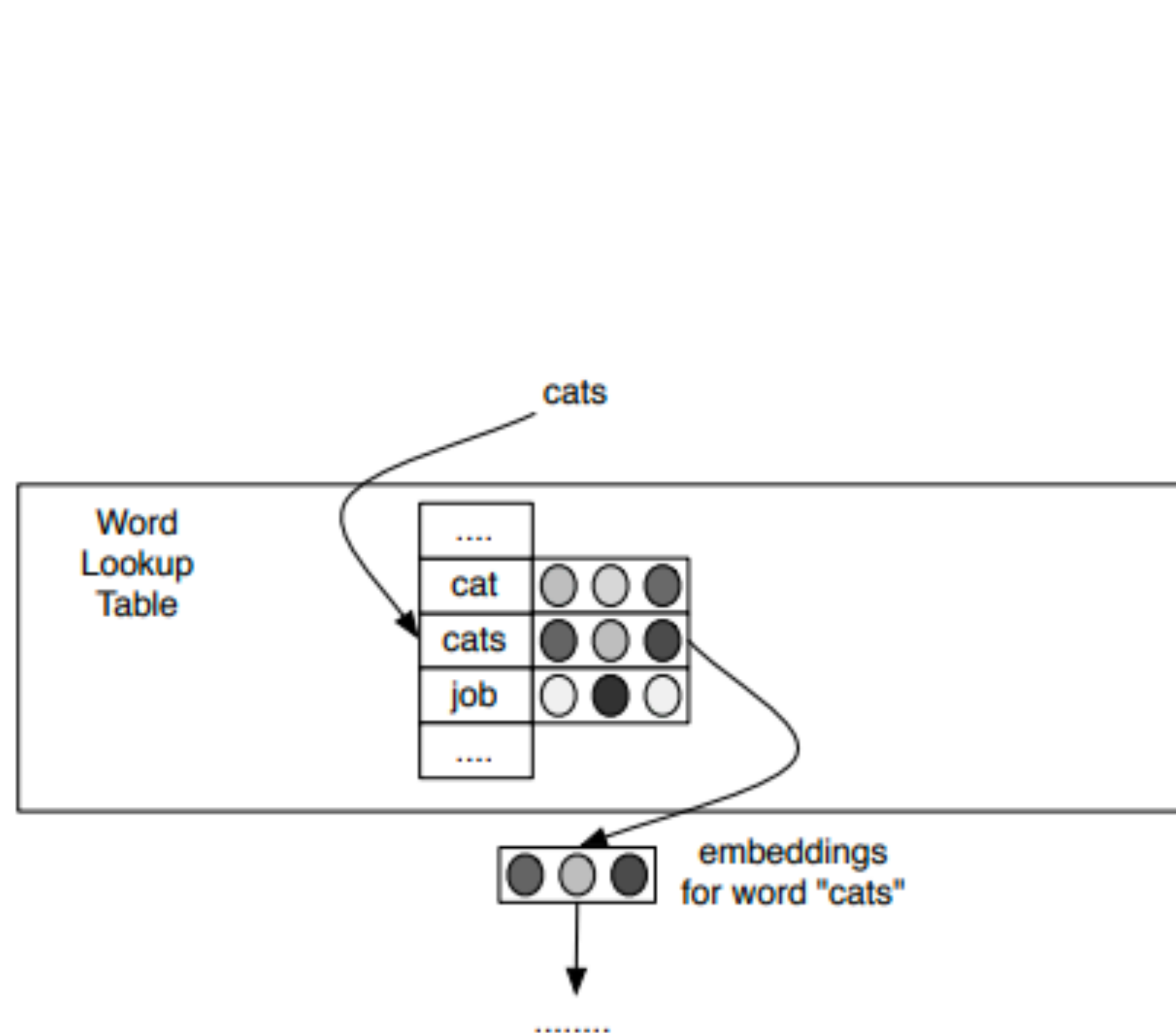
I saw the dog running in the park and it looked very excited to be there

batch dim



- ▶ Multiple sequences **and** multiple timestamps per sequence

Subword Models



Ling et al. (2015)

Padding

- ▶ Prepending or appending zeros
- ▶ To create batches of equal length for faster training time

▶ Padding for character-level LSTM

Adylov [2 7 13 8 10 12]

Sloan [4 8 10 5 9]

Harb [3 5 11 6]

San [4 5 9]



Adylov [2 7 13 8 10 12]

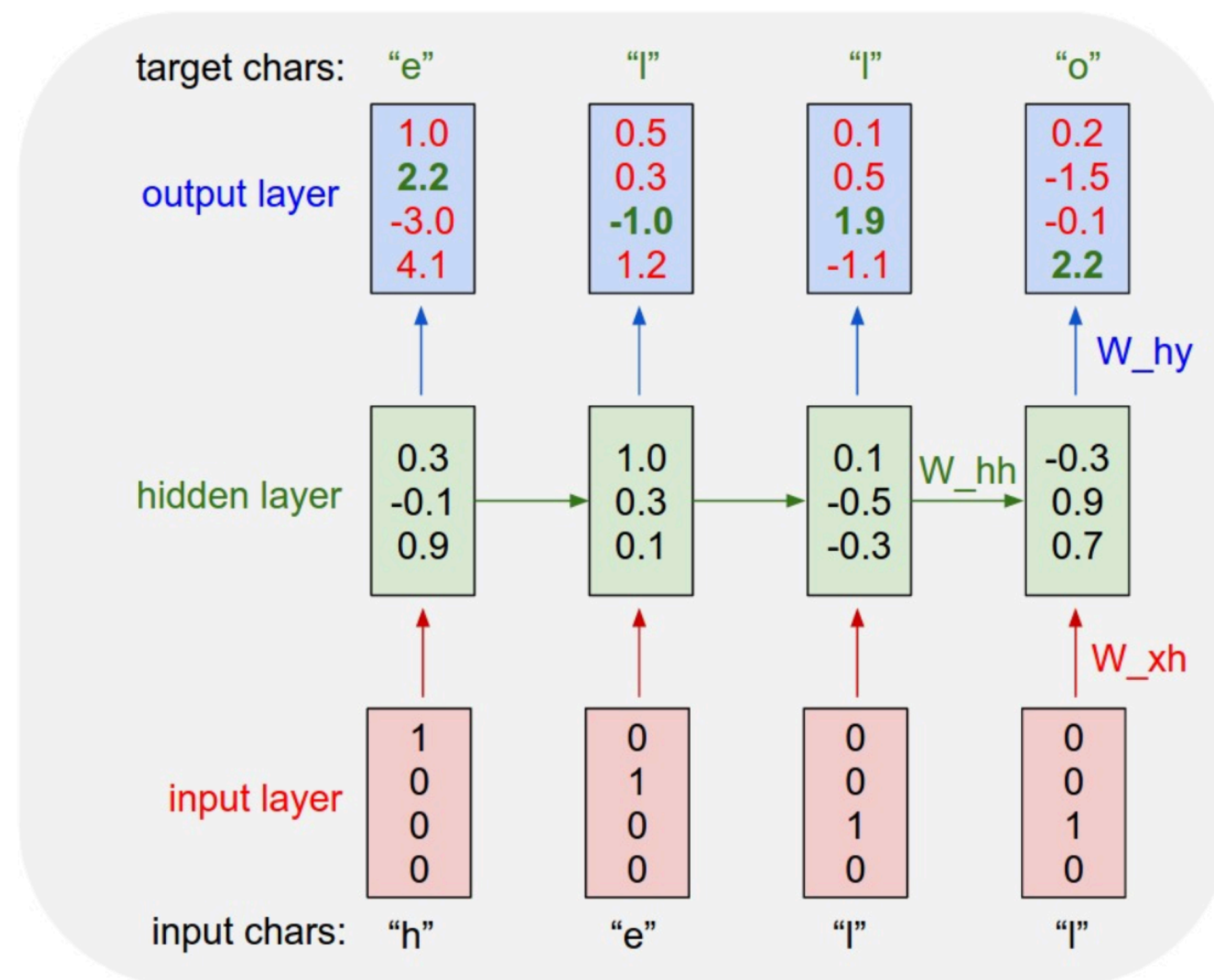
Sloan [0 4 8 10 5 9]

Harb [0 0 3 5 11 6]

San [0 0 0 4 5 9]

Visualizing LSTMs

- ▶ Train *character* LSTM language model (predict next character based on history) over two datasets: War and Peace and Linux kernel source code



An example RNN with 4-dimensional input and output layers, and a hidden layer of 3 units (neurons). This diagram shows the activations in the forward pass when the RNN is fed the characters "hell" as input. The output layer contains confidences the RNN assigns for the next character (vocabulary is "h,e,l,o"); We want the green numbers to be high and red numbers to be low.

Visualizing LSTMs

- ▶ Train *character* LSTM language model (predict next character based on history) over two datasets: War and Peace and Linux kernel source code
- ▶ Visualize activations of specific cells (components of **c**) to understand them
- ▶ Counter: know when to generate \n

The sole importance of the crossing of the Berezina lies in the fact that it plainly and indubitably proved the fallacy of all the plans for cutting off the enemy's retreat and the soundness of the only possible line of action--the one Kutuzov and the general mass of the army demanded--namely, simply to follow the enemy up. The French crowd fled at a continually increasing speed and all its energy was directed to reaching its goal. It fled like a wounded animal and it was impossible to block its path. This was shown not so much by the arrangements it made for crossing as by what took place at the bridges. When the bridges broke down, unarmed soldiers, people from Moscow and women with children who were with the French transport, all--carried on by vis inertiae--pressed forward into boats and into the ice-covered water and did not, surrender.

Visualizing LSTMs

- ▶ Train *character* LSTM language model (predict next character based on history) over two datasets: War and Peace and Linux kernel source code
- ▶ Visualize activations of specific cells to see what they track
- ▶ Binary switch: tells us if we're in a quote or not

"You mean to imply that I have nothing to eat out of.... On the contrary, I can supply you with everything even if you want to give dinner parties," warmly replied Chichagov, who tried by every word he spoke to prove his own rectitude and therefore imagined Kutuzov to be animated by the same desire.

Kutuzov, shrugging his shoulders, replied with his subtle penetrating smile: "I meant merely to say what I said."

Visualizing LSTMs

- ▶ Train *character* LSTM language model (predict next character based on history) over two datasets: War and Peace and Linux kernel source code
- ▶ Visualize activations of specific cells to see what they track
- ▶ Stack: activation based on indentation

```
#ifdef CONFIG_AUDITSYSCALL
static inline int audit_match_class_bits(int class, u32 *mask)
{
    int i;
    if (classes[class]) {
        for (i = 0; i < AUDIT_BITMASK_SIZE; i++)
            if (mask[i] & classes[class][i])
                return 0;
    }
    return 1;
}
```

Visualizing LSTMs

- ▶ Train *character* LSTM language model (predict next character based on history) over two datasets: War and Peace and Linux kernel source code
- ▶ Visualize activations of specific cells to see what they track
- ▶ Uninterpretable: probably doing double-duty, or only makes sense in the context of another activation

```
/* Unpack a filter field's string representation from user-space
 * buffer. */
char *audit_unpack_string(void **bufp, size_t *remain, size_t len)
{
    char *str;
    if (!*bufp || (len == 0) || (len > *remain))
        return ERR_PTR(-EINVAL);
    /* Of the currently implemented string fields, PATH_MAX
     * defines the longest valid length.
     */
}
```

Takeaways

- ▶ RNNs can transduce inputs (produce one output for each input) or compress the whole input into a vector
- ▶ Useful for a range of tasks with sequential input: sentiment analysis, natural language inference, machine translation, language modeling